

人工智能期末复习课详细提纲

袁肖赞，上海交通大学计算机学院，yuanxiaoyun@sjtu.edu.cn

Contents

1	基础知识	3
1.1	张量与形状	3
1.2	参数、权重与 bias	3
1.3	激活函数	4
1.4	Softmax 与概率输出	4
1.5	交叉熵	5
1.6	关键记忆	6
1.7	思考题	6
2	全连接网络与残差网络基础	6
2.1	单个神经元与权重	6
2.2	全连接层	7
2.3	多层神经网络与激活函数	8
2.4	深层网络训练困难	8
2.5	残差连接	9
2.6	关键记忆	10
2.7	思考题	10
3	卷积神经网络 CNN	10
3.1	图像数据的特征：关联性	10
3.2	图像数据的特征：不变性	11
3.3	大脑如何处理图像：类视皮层设计	11
3.4	卷积运算	11
3.5	Zero Padding	12
3.6	Stride	13
3.7	多通道卷积	15
3.8	卷积层尺寸计算总结	15
3.9	池化 Pooling	17
3.10	CNN 整体结构与层的组合	17
3.11	关键记忆	18
3.12	思考题	18
4	循环神经网络 RNN 与 LSTM	18
4.1	自然语言处理 NLP 的任务特点	18
4.2	数据准备	19
4.3	分词、one-hot 与 embedding	20
4.4	Next Token Prediction	21
4.5	RNN 的设计	22
4.6	BPTT 与梯度问题	24
4.7	LSTM	25
4.8	关键记忆	26
4.9	思考题	26
5	Transformer	26

5.1	为什么需要 Transformer	26
5.2	Next Token Prediction 回顾	28
5.3	Embedding 与位置编码	28
5.4	注意力机制与 Q、K、V	29
5.5	Causal Mask	30
5.6	Transformer Block、LayerNorm 与 FNN	31
5.7	输出投影与解码	32
5.8	关键记忆	34
5.9	思考题	34
6	神经网络训练现象：频率原则	34
6.1	从训练现象出发	34
6.2	泛化谜团与隐式偏好	35
6.3	频率、低频与高频	36
6.4	频率原则	37
6.5	频率原则与泛化	38
6.6	Early Stopping 早停	38
6.7	图像分类中的两种频率	39
6.8	关键记忆	40
6.9	思考题	40
7	参数凝聚与解的平坦性	41
7.1	初始化大小如何影响训练	41
7.2	ReLU 神经元与转折点	41
7.3	参数凝聚	42
7.4	为什么不直接训练小网络	43
7.5	平坦解与尖锐解	44
7.6	提升平坦性的常见技巧	44
7.7	关键记忆	45
7.8	思考题	45
8	大模型介绍	46
8.1	什么是大模型	46
8.2	Encoder、Decoder 与常见架构	46
8.3	预训练、微调、提示词、蒸馏	46
8.4	大模型中的现象	47
8.5	思考题	47
9	强化学习	47
9.1	强化学习是什么	47
9.2	状态、观察、动作空间、奖励	48
9.3	回报与折扣因子	48
9.4	情节型任务与连续型任务	48
9.5	探索与利用	49
9.6	策略、价值函数与最优策略	49
9.7	MDP 与马尔可夫性	49
9.8	Bellman 方程的直观含义	50
9.9	DP、MC 与深度强化学习	50
9.10	强化学习应用	51
9.11	思考题	51
10	最后一遍串讲清单	51
10.1	残差网络必会	51
10.2	CNN 必会	51
10.3	RNN/LSTM 必会	51
10.4	Transformer 必会	52
10.5	训练现象必会	52

10.6 大模型必会	52
10.7 强化学习必会	52
11 不建议作为考试重点的内容	53
12 两节复习课建议讲法	53
12.1 第一节课：网络结构主线	53
12.2 第二节课：训练现象、大模型、强化学习	53

1 基础知识

这一章用于补齐后面章节会反复出现的基础概念：张量形状、参数与 bias、激活函数、softmax 和交叉熵。这里不讲训练过程中的前向传播和反向传播。

1.1 张量与形状

神经网络里的数据通常不是单个数字，而是向量、矩阵或更高维数组。复习时最重要的是看懂“形状”。

张量 Tensor：张量可以理解为多维数组；标量是 0 维，向量是 1 维，矩阵是 2 维，图像和特征图通常是 3 维或 4 维。

常见形状记法：

数据	常见形状	含义
向量	d	有 d 个数
矩阵	$m \times n$	m 行、 n 列
图像	$H \times W \times C$	高、宽、通道数
一批样本	$B \times H \times W \times C$	batch size、高、宽、通道数
token 序列	$n \times d_m$	序列长度、每个 token 的向量维度

例子 1：图像形状。一张 RGB 图像大小为 32×32 ，有 3 个颜色通道，因此形状可以写成 $32 \times 32 \times 3$ 。

例子 2：token 序列形状。一个句子有 $n = 4$ 个 token，每个 token 的 embedding 维度为 $d_m = 128$ ，则输入矩阵形状为 4×128 。

1.2 参数、权重与 bias

神经网络中需要学习的数通常叫参数。最常见的参数是权重和 bias。

参数：参数是神经网络中需要通过数据学习得到的数，例如全连接层的权重矩阵 W 和 bias 向量 b 。

Bias：Bias 也叫偏置项，它是在线性变换 $Wx + b$ 中额外加上的可学习参数。

全连接层常写成：

$$y = Wx + b$$

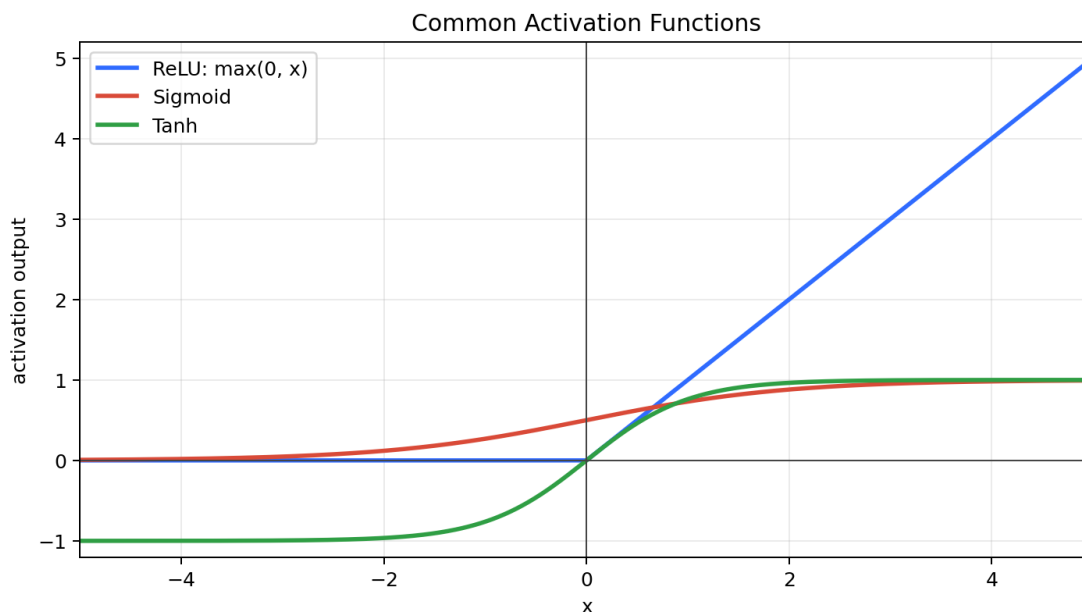
如果输入维度为 d_{in} ，输出维度为 d_{out} ，则：

$$W \in R^{d_{out} \times d_{in}}, \quad b \in R^{d_{out}}, \quad y \in R^{d_{out}}$$

例子 3：全连接层参数量。输入维度为 4，输出维度为 3。权重参数数量为 $3 \times 4 = 12$ ，bias 参数数量为 3，总参数数量为 15。

1.3 激活函数

激活函数把线性变换后的结果变成非线性输出。没有非线性激活函数，多层线性变换叠在一起仍然等价于一个线性变换。



激活函数：激活函数是作用在神经元输出上的非线性函数，用来提升神经网络表达复杂关系的能力。

常见激活函数：

激活函数	公式	记忆
ReLU	$\text{ReLU}(x) = \max(0, x)$	负数变 0，正数保留
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	输出在 0 到 1 之间
Tanh	$\tanh(x)$	输出在 -1 到 1 之间

例子 4：ReLU 计算。ReLU(-2) = 0，ReLU(3) = 3。

例子 5：Sigmoid 的用途。LSTM 的门常用 sigmoid，因为 sigmoid 输出在 0 到 1 之间，适合表示“保留多少”“写入多少”这种比例。

1.4 Softmax 与概率输出

分类任务和语言模型输出时，模型通常先给每个类别或 token 一个分数，再把这些分数变成概率。

Softmax：Softmax 把一组分数转换成概率分布；输出非负，并且所有概率之和为 1。

Softmax 公式为：

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax 会放大分数之间的差异：大的分数对应更大的概率，小的分数对应更小的概率，输出会比原始分数更接近 one-hot 分布。

例子 6：Softmax 把分数变成概率。假设两个类别的分数为 $[1, 2]$ ，则

$$e^1 \approx 2.72, \quad e^2 \approx 7.39$$

所以 softmax 后的概率约为：

$$\left[\frac{2.72}{2.72 + 7.39}, \frac{7.39}{2.72 + 7.39} \right] \approx [0.27, 0.73]$$

分数更大的类别概率更大，小分数对应的类别概率更小；结果比原始分数更像 one-hot，但两个概率之和仍然为 1。

1.5 交叉熵

交叉熵常用于分类任务和 next token prediction。复习时只需要知道：真实类别的预测概率越高，交叉熵越小。

交叉熵：交叉熵衡量真实标签和模型预测概率之间的差距；对真实类别给出的概率越高，交叉熵越小。

如果真实类别是第 k 类，模型给第 k 类的预测概率为 p_k ，则单个样本的交叉熵可以写成：

$$L = -\log p_k$$

二分类形式：

如果真实标签 $y \in \{0, 1\}$ ，模型预测正类概率为 p ，则二分类交叉熵为：

$$L = -[y \log p + (1 - y) \log(1 - p)]$$

当 $y = 1$ 时， $L = -\log p$ ；当 $y = 0$ 时， $L = -\log(1 - p)$ 。

多分类形式：

如果真实标签用 one-hot 向量 y_i 表示，模型预测概率为 p_i ，则多分类交叉熵为：

$$L = -\sum_i y_i \log p_i$$

因为 one-hot 里只有真实类别对应的 $y_i = 1$ ，所以多分类形式也会退化成 $L = -\log p_k$ 。

例子 7：交叉熵计算。对二分类任务，真实标签是第 2 类，也就是 one-hot 标签 $y = [0, 1]$ 。如果模型预测为 $p = [0.1, 0.9]$ ，交叉熵为：

$$L = -(0 \cdot \log 0.1 + 1 \cdot \log 0.9) = -\log 0.9 \approx 0.105$$

如果模型预测为 $p = [0.4, 0.6]$ ，交叉熵为：

$$L = -(0 \cdot \log 0.4 + 1 \cdot \log 0.6) = -\log 0.6 \approx 0.511$$

因为 0.9 比 0.6 更接近真实类别，所以第一种预测的交叉熵更小。

1.6 关键记忆

- 看神经网络结构时，先看清输入和输出的形状。
- 参数是网络中需要学习的数，常见参数包括权重 W 和 bias b 。
- 激活函数提供非线性能力；ReLU 的公式是 $\max(0, x)$ 。
- Sigmoid 输出在 0 到 1 之间，常用于表示比例或门控。
- Softmax 把分数变成概率分布，所有概率之和为 1；大的更大、小的更小，结果更接近 one-hot。
- 交叉熵中，真实类别的预测概率越高，loss 越小；单个样本可记为 $L = -\log p_k$ 。

1.7 思考题

1. 填空：一张 32×32 的 RGB 图像，形状可以写成 $32 \times 32 \times \underline{\quad}$ 。
答案：3。
2. 判断：Bias 是神经网络中的可学习参数。
答案：正确。
3. 计算：ReLU(-5) 和 ReLU(4) 分别是多少？
答案：0 和 4。
4. 判断：Softmax 输出的所有概率之和为 1。
答案：正确。
5. 计算：真实类别概率为 0.9 时，单个样本交叉熵可以写成 $-\log \underline{\quad}$ 。
答案：0.9。

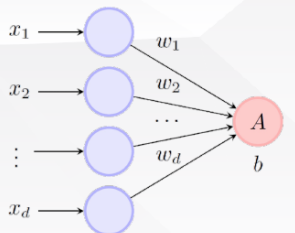
2 全连接网络与残差网络基础

这一章按 PDF 顺序复习：单个神经元 -> 全连接层 -> 多层神经网络 -> 深层网络训练困难 -> 残差连接。第 11-12 节中，全连接网络只保留理解后续章节所需的基础；复习重点放在残差网络为什么出现、残差连接怎么写、它解决什么问题。

2.1 单个神经元与权重

页码：p4-p6

单个神经元可以理解为“收集多个输入，按重要性加权，再经过激活函数输出”。权重越大，说明对应输入对输出影响越大。



$x_1 \rightarrow$
 $x_2 \rightarrow$
 $\vdots$
 $x_d \rightarrow$

w_1
 w_2
 $\dots$
 w_d

A
 b

$A = \sum_{i=1}^d w_i x_i + b$

$w_1 x_1 + \dots + w_d x_d + b$
 $= (w_1, \dots, w_d) \cdot (x_1, \dots, x_d) + b$
 $= \mathbf{w}^T \mathbf{x} + b.$

单个神经元的输出为 $\sigma(\mathbf{w}^T \mathbf{x} + b)$ ， $\sigma(\cdot)$ 通常叫做激活函数。

$$\gamma(x) = \begin{cases} 1 & x \geq T, \\ 0 & x < T. \end{cases}$$

神经元：神经元把输入按权重加权求和，加上 bias，再通过激活函数得到输出。

单个神经元公式：

$$z = w^T x + b, \quad y = \sigma(z)$$

其中， x 是输入向量， w 是权重向量， b 是 bias， σ 是激活函数， y 是神经元输出。

例子 1：单个神经元计算。设 $x = [2, 1]^T$ ， $w = [0.5, -1]^T$ ， $b = 0.2$ ，则

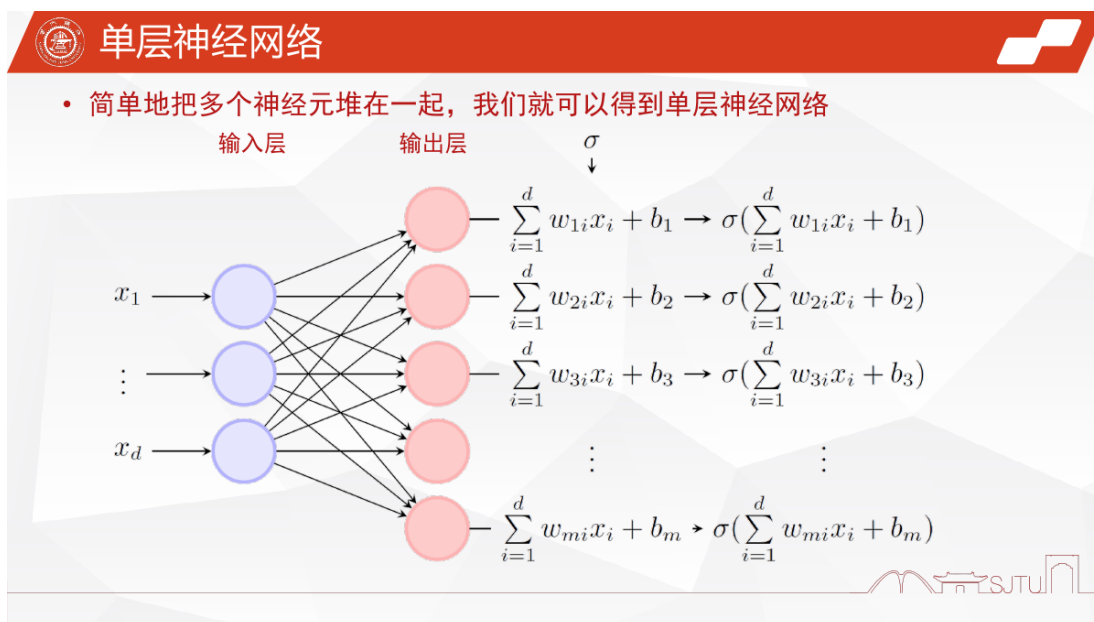
$$z = 0.5 \times 2 + (-1) \times 1 + 0.2 = 0.2$$

神经元最后输出为 $y = \sigma(0.2)$ 。如果 σ 是 ReLU，则 $y = \max(0, 0.2) = 0.2$ 。

2.2 全连接层

页码：p7-p8

把多个神经元放在同一层，并让每个输出神经元都连接到所有输入，就得到全连接层。全连接层的核心是矩阵乘法。



全连接层：全连接层中，每个输出神经元都接收上一层所有输入；它可以写成矩阵形式 $y = \sigma(Wx + b)$ 。

全连接层公式：

$$y = \sigma(Wx + b)$$

如果输入维度是 d_{in} ，输出维度是 d_{out} ，则

$$x \in R^{d_{in}}, \quad W \in R^{d_{out} \times d_{in}}, \quad b \in R^{d_{out}}, \quad y \in R^{d_{out}}$$

例子 2：全连接层参数量。输入维度 $d_{in} = 4$ ，输出维度 $d_{out} = 3$ 。权重矩阵 W 有 $3 \times 4 = 12$ 个参数，bias 有 3 个参数，所以总参数量为 $12 + 3 = 15$ 。

2.3 多层神经网络与激活函数

页码：p10-p18

多层神经网络是在全连接层基础上继续堆叠隐藏层。只做线性变换不够，因为很多任务本身是非线性的；加入非线性激活函数后，网络才能表达更复杂的函数。

多层神经网络 **MLP**：MLP 由多层全连接层和非线性激活函数堆叠而成，常见形式是“线性变换 + 激活函数”重复多次。

一层隐藏层的常见形式：

$$h = \sigma(W_1 x + b_1), \quad y = W_2 h + b_2$$

其中， h 是隐藏层表示。考试重点不是复杂推导，而是知道：激活函数让网络具有非线性表达能力。

例子 3：为什么需要激活函数。如果连续两层都只是线性变换，那么

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$$

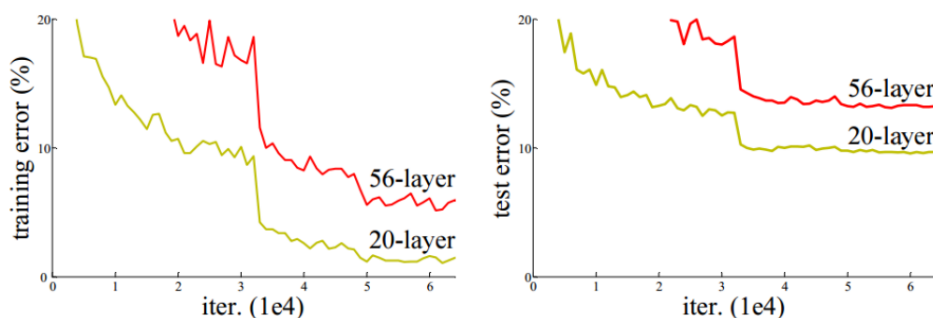
整体仍然只是一个线性变换。加入非线性激活函数后，多层网络才不容易退化成单层线性模型。

2.4 深层网络训练困难

页码：p19-p21

网络变深后，理论表达能力更强，但实际训练不一定更好。普通深层网络可能出现梯度消失、梯度爆炸和退化问题。

退化问题



利用20层和56层的普通网络对于CIFAR-10数据集的训练误差（左图）和测试误差（右图）。可见越深的网络会产生越大的训练误差和测试误差。

[1] He, Kaiming, et al. "Deep Residual Learning for Image Recognition." IEEE (2016).



退化问题：当网络加深后，训练误差和测试误差没有变好，反而可能变差，这说明深层普通网络更难优化。

例子 5：残差输出计算。如果输入 $x = [1, 2]^T$ ，网络学习到的变化是 $F(x) = [0.2, -0.1]^T$ ，则残差模块输出为

$$F(x) + x = [0.2, -0.1]^T + [1, 2]^T = [1.2, 1.9]^T$$

残差连接的意义不是让网络“少学一点”，而是让网络更容易学习恒等映射。如果某些额外层暂时学不到有用特征，它们可以让 $F(x)$ 接近 0，此时输出接近 x ，网络不会因为加深而明显变差。

例子 6：恒等映射。如果一个残差模块暂时没有学到有效变化，令 $F(x) \approx 0$ ，则输出 $F(x) + x \approx x$ 。这说明深层网络至少可以比较容易地保留原来的信息。

2.6 关键记忆

- 单个神经元的基本计算是 $z = w^T x + b$ ，再经过激活函数得到输出。p4-p6
- 全连接层可以写成 $y = \sigma(Wx + b)$ ；如果输入维度为 d_{in} 、输出维度为 d_{out} ，则 $W \in R^{d_{out} \times d_{in}}$ 。p7-p8
- 激活函数提供非线性能力；如果多层网络只有线性变换，整体仍然等价于一个线性变换。p10-p18
- 深层普通网络可能遇到梯度消失、梯度爆炸和退化问题；网络更深不一定更容易训练。p19-p21
- 残差连接的基本形式是 $F(x) + x$ ，它帮助信息和梯度在深层网络中传播。p22-p23
- ResNet 主要缓解深层网络训练困难和退化问题，不是单纯用于减少参数或解决过拟合。p22-p23

2.7 思考题

1. 填空：单个神经元常见计算形式是 $z = w^T x + b$ ，其中 b 叫做 _____。
答案：bias，也叫偏置项。
2. 判断：如果多层网络中每一层都只有线性变换，那么多层叠加后整体仍然等价于一个线性变换。
答案：正确。
3. 计算：输入维度为 5、输出维度为 2 的全连接层，如果包含 bias，总参数量是多少？
答案： $2 \times 5 + 2 = 12$ 。
4. 填空：残差连接的基本形式可以写成 $F(x) + \underline{\quad}$ 。
答案： x 。
5. 判断：残差连接本身的核心作用是让信息和梯度更容易在深层网络中传播。
答案：正确。

3 卷积神经网络 CNN

这一章按 PDF 顺序复习：图像特性 -> 类视皮层启发 -> 卷积计算 -> padding -> stride -> 多通道卷积 -> 池化 -> CNN 结构。页码均指本章 PDF 页码。

3.1 图像数据的特征：关联性

页码：p3-p5

图像有空间结构。相邻像素通常更相关，距离越远，关联通常越弱。

图像局部关联性：图像中距离较近的像素或局部区域通常更相关，距离越远，相关性通常越弱。CNN 的局部连接设计正是为了利用这种特点。

如果两个像素点完全独立、互不影响，那么它们的相关函数或互信息量为 0。这个结论常用于判断“两个位置是否还有统计关联”。

模型	形式	记忆
指数衰减	$y = e^{-kx}$	下降较快

模型	形式	记忆
幂级数衰减	$y = \frac{1}{x^k}$	下降较慢，长程关联更明显

幂级数衰减可以看作许多指数衰减成分的叠加：

$$\int_0^\infty e^{-\alpha t} d\alpha = \frac{1}{t}$$

3.2 图像数据的特征：不变性

页码：p6

图像分类希望模型识别“物体是什么”，而不是死记每个像素的绝对位置。因此，物体发生小幅平移、旋转、伸缩或光照变化时，类别通常不应改变。

图像不变性：图像通常具有平移不变性、旋转不变性和伸缩不变性；这些变化不应轻易改变图像类别。

不是置换不变：图像通常不具有置换不变性；如果把像素顺序完全打乱，空间结构会被破坏，识别结果也会受到影响。

3.3 大脑如何处理图像：类视皮层设计

页码：p7-p10

图像处理可以理解为：先提取局部简单特征，再逐步整合成复杂信息。

类比对象	作用	CNN 中的对应模块
简单细胞	对位置、边缘、方向敏感	卷积层
复杂细胞	整合多个局部输入	池化、全连接等后续模块

CNN 模块	作用
卷积层	提取局部特征
池化层	汇总局部信息
全连接层	综合特征
输出层	输出类别或概率

3.4 卷积运算

页码：p11-p13

卷积核是一个小窗口，例如 3×3 。它在图像上滑动，每次覆盖一个局部区域，把局部区域和卷积核对应位置相乘后求和。

卷积运算：卷积核在输入图像或特征图上滑动，每次对局部窗口做对应位置相乘再求和，得到输出特征图中的一个值。

卷积核中的权重通常是随机初始化后通过训练学习得到的，并不是固定不变的手工模板。卷积层的 bias 也是可学习参数；每个输出通道通常对应一个 bias 标量。

卷积运算



图像

右边的例子中用的是 $g(-u, -v)$, 是否等价?

卷积核

是等价, 本质上, 做个变量代换即可。

$$F(x, y) = \sum_{u, v} f(x - u, y - v) g(u, v)$$

$$F(x, y) = \int f(x - u, y - v) g(u, v) du dv$$

$g(-1, 1)$	$g(0, 1)$	$g(1, 1)$
$g(-1, 0)$	$g(0, 0)$	$g(1, 0)$
$g(-1, -1)$	$g(0, -1)$	$g(1, -1)$

	$(x-1, y+1)$	$(x, y+1)$	$(x+1, y+1)$	
	$(x-1, y)$	(x, y)	$(x+1, y)$	
	$(x-1, y-1)$	$(x, y-1)$	$(x+1, y-1)$	

$$F(x, y) = f(x-1, y+1)g(-1, 1) + f(x, y+1)g(0, 1) + f(x+1, y+1)g(1, 1) \\ + f(x-1, y)g(-1, 0) + f(x, y)g(0, 0) + f(x+1, y)g(1, 0) \\ + f(x-1, y-1)g(-1, -1) + f(x, y-1)g(0, -1) + f(x+1, y-1)g(1, -1)$$

[1] 《深度学习现象导论》许志钦、张耀宇 https://github.com/xuzhiqin1990/understanding_dl



单通道卷积例子:
图像局部区域 X :

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 2 \\ 0 & 1 & 1 \end{bmatrix}$$

卷积核 K :

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

一次卷积输出为

$$1 \times 1 + 2 \times 0 + 0 \times (-1) + 3 \times 1 + 1 \times 0 + 2 \times (-1) + 0 \times 1 + 1 \times 0 + 1 \times (-1) = 1$$

卷积核继续向右或向下滑动, 就得到下一个位置的输出。所有位置的输出组成新的特征图。

模型	连接方式	对图像结构的利用
MLP	常把图像展平成向量, 全连接	容易弱化二维空间结构
CNN	局部连接, 卷积核滑动	直接利用局部关联

3.5 Zero Padding

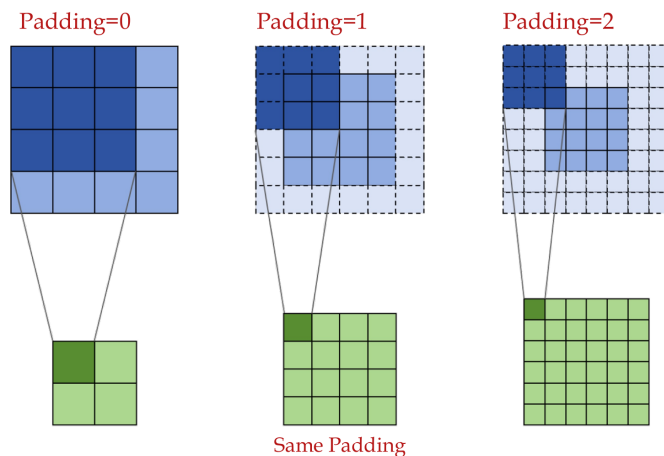
页码: p14

Padding 是在图像边缘补值, 常见是补 0。作用是控制卷积后图像大小, 避免图像尺寸过快变小。

Padding: Padding 是在输入边缘补值, 常见是补 0, 用来控制卷积后的空间尺寸, 并让边缘位置也能参与更多卷积计算。

Same Padding 指通过合适的 padding 让输出空间尺寸和输入空间尺寸相同。填充本身只是补值, 不会引入可学习参数。

Zero Padding——解决图像过卷积后大小改变的问题



[1] 《深度学习现象导论》许志钦、张耀宇 https://github.com/xuzhiqin1990/understanding_dl



Zero padding 例子：原图像为 3×3 ：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

padding 为 1 后变成 5×5 ：

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 0 \\ 0 & 7 & 8 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

使用 3×3 卷积核、stride=1 时：

不加 padding: $3 \times 3 \rightarrow 1 \times 1$ 。加 padding 为 1: $3 \times 3 \rightarrow 3 \times 3$ 。

3.6 Stride

页码: p15

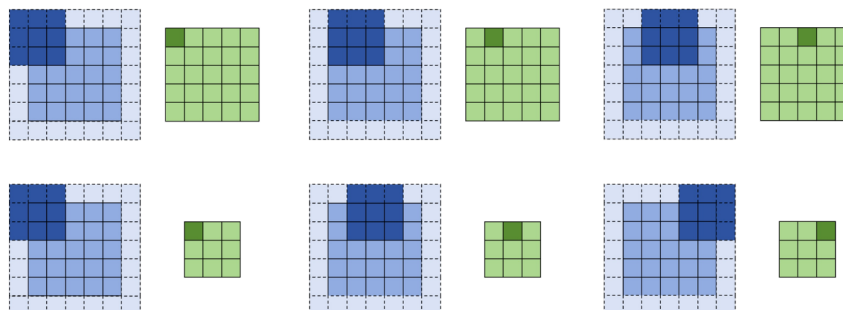
Stride 是卷积核滑动步长。课件用 $\text{stride}(i, j)$ 表示横向跨 i 步、纵向跨 j 步。

Stride: Stride 是卷积核每次滑动的步长。stride 越大，卷积核采样位置越少，输出空间尺寸通常越小。

跨步 stride(i, j)



横向跨i步，纵向跨j步。图中第一行为stride(1,1)，第二行为stride(2,2)



[1] 《深度学习现象导论》许志钦、张耀宇 https://github.com/xuzhiqin1990/understanding_dl



Stride 例子：输入长度方向位置为 1, 2, 3, 4, 5，卷积核大小为 3。
stride 为 1 时，窗口为 [1, 2, 3]、[2, 3, 4]、[3, 4, 5]，输出长度为 3。
stride 为 2 时，窗口为 [1, 2, 3]、[3, 4, 5]，输出长度为 2。

3.7 多通道卷积

页码: p16

RGB 图像有 3 个输入通道。多通道卷积中，一个完整卷积核需要覆盖所有输入通道。

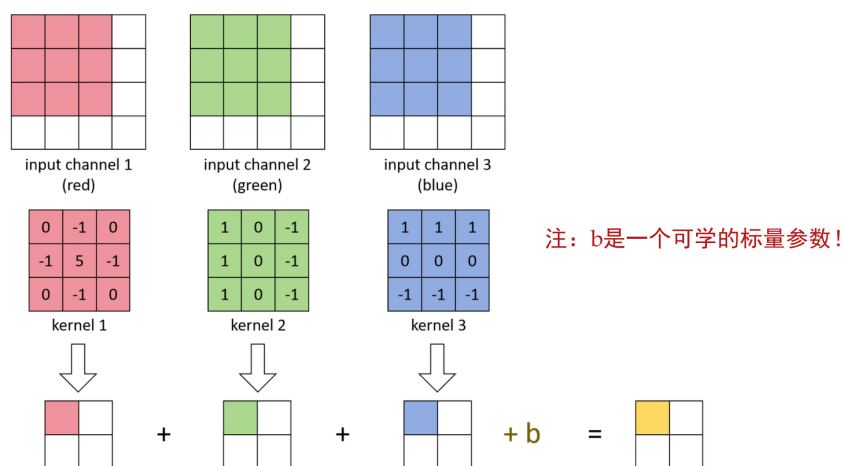
多通道卷积：对于有 C_{in} 个输入通道的数据，一个完整卷积核的深度也必须覆盖 C_{in} 个通道；卷积核个数通常记为 C_{out} ，也就是输出通道数。

单个输出通道的计算：

$$Y_j = \sum_{c=1}^{C_{in}} X_c * K_{j,c} + b_j$$

其中， X_c 是第 c 个输入通道， $K_{j,c}$ 是第 j 个卷积核在第 c 个输入通道上的部分， b_j 是第 j 个输出通道的 bias。对 $j = 1, \dots, C_{out}$ 都计算一次，就得到 C_{out} 个输出通道。

多通道卷积



[1] 《深度学习现象导论》许志钦、张耀宇 https://github.com/xuzhiqin1990/understanding_dl



多通道卷积例子：RGB 输入图像有 3 个通道。R 通道局部区域与卷积核 R 部分相乘求和，G 通道局部区域与卷积核 G 部分相乘求和，B 通道局部区域与卷积核 B 部分相乘求和；三个通道结果相加，再加 bias，就是这个卷积核在当前位置的输出。

通道数例子：输入为 $32 \times 32 \times 3$ ，一个完整卷积核大小为 $5 \times 5 \times 3$ 。如果卷积核个数为 6，则输出空间大小另算，输出通道数为 6。

3.8 卷积层尺寸计算总结

页码: p14-p16

卷积层的计算重点记三件事：空间大小怎么变，通道数怎么变，参数量怎么算。

卷积层输出形状：输入为 $H \times W \times C_{in}$ ，卷积核个数为 C_{out} 时，输出形状为 $H_{out} \times W_{out} \times C_{out}$ ；输出通道数由卷积核个数决定。

输入、卷积核和输出的形状：

输入: $H \times W \times C_{\text{in}}$

一个卷积核: $K \times K \times C_{\text{in}}$, 卷积核个数: C_{out}

输出: $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$

符号含义: H, W 是输入高和宽; C_{in} 是输入通道数; K 是卷积核大小, 例如 3×3 卷积核中 $K = 3$; C_{out} 是卷积核个数, 也就是输出通道数; $H_{\text{out}}, W_{\text{out}}$ 是输出高和宽。

输出空间尺寸公式:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

其中, P 是 padding 大小, S 是 stride 大小, $\lfloor \cdot \rfloor$ 表示向下取整。

padding、stride、卷积核大小的影响:

参数	增大后通常会怎样	直观理解
padding	输出空间尺寸变大	边缘补了一圈, 能滑动的位置更多
stride	输出空间尺寸变小	卷积核跳得更远, 采样位置更少
卷积核大小	输出空间尺寸变小	窗口更大, 可放置的位置更少
卷积核个数	输出通道数变多	一个卷积核产生一个输出通道

参数量计算:

卷积层通常有 bias。每个卷积核产生一个输出通道, 因此每个输出通道对应 1 个 bias。

不考虑 bias:

$$\# \text{params} = K \times K \times C_{\text{in}} \times C_{\text{out}}$$

考虑 bias:

$$\# \text{params} = K \times K \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$$

例子 1: 参数量计算。输入通道数 $C_{\text{in}} = 3$, 卷积核大小 $K = 5$, 卷积核个数 $C_{\text{out}} = 6$ 。不考虑 bias 时, 参数量为 $5 \times 5 \times 3 \times 6 = 450$; 考虑 bias 时, 参数量为 $5 \times 5 \times 3 \times 6 + 6 = 456$ 。

例子 2: padding 保持大小。输入为 $32 \times 32 \times 3$, 使用 3×3 卷积核, 卷积核个数为 16, padding 为 1, stride 为 1。

$$H_{\text{out}} = \frac{32 + 2 \times 1 - 3}{1} + 1 = 32$$

因此输出为 $32 \times 32 \times 16$ 。

例子 3: **stride** 让尺寸减小。输入为 $32 \times 32 \times 3$, 使用 3×3 卷积核, 卷积核个数为 16, padding 为 1, stride 为 2。

$$H_{\text{out}} = \left\lfloor \frac{32 + 2 \times 1 - 3}{2} \right\rfloor + 1 = 16$$

因此输出为 $16 \times 16 \times 16$ 。

3.9 池化 Pooling

页码: p18

池化层用于对局部区域做汇总, 常见方式是最大池化和平均池化。池化通常用于降低空间尺寸、减少计算量, 一般没有可学习参数, 也通常不改变通道数。

池化 Pooling: 池化是在局部窗口内做汇总操作。最大池化取窗口最大值, 保留最强响应; 平均池化取窗口平均值, 保留整体平均水平。

全局平均池化 GAP: GAP 对每个通道的所有空间位置取平均, 把每个通道压成一个数; 它改变空间尺寸, 但不改变通道数。

GAP 常用于减少后续全连接层的参数量。例如输入为 $7 \times 7 \times 512$, GAP 后变成 $1 \times 1 \times 512$, 每个通道只保留一个平均值。

池化例子: 对局部区域

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

做最大池化时, 输出为 $\max(1, 3, 2, 4) = 4$; 做平均池化时, 输出为 $\frac{1+3+2+4}{4} = 2.5$ 。

尺寸例子: 输入为 $32 \times 32 \times 16$, 最大池化窗口为 2×2 , stride 为 2, 输出为 $16 \times 16 \times 16$ 。

3.10 CNN 整体结构与层的组合

页码: p20-p22

p20 给出了一个典型 CNN 结构:

输入 RGB 三通道图像

-> 卷积 + ReLU + 池化

-> 卷积 + ReLU + 池化

-> 展平

-> 全连接

-> Softmax 输出类别概率

经典 **LeNet** 结构: 输入图像 -> 卷积 -> 池化 -> 卷积 -> 池化 -> 展平 -> 全连接 -> 输出。

需要注意: CNN 没有唯一固定的网络格式。卷积层、激活函数、池化层、全连接层等都可以按任务需要组合。

常见组合方式:

卷积 -> 激活 -> 卷积 -> 激活 -> 池化

卷积 -> 激活 -> 池化 -> 卷积 -> 激活 -> 池化

卷积 -> 激活 -> 展平 -> 全连接 -> 输出

只要前一层的输出形状能作为后一层的输入，这些层就可以灵活搭配。不同 CNN 结构的差别，往往就体现在层的数量、顺序、卷积核个数、是否使用池化等设计上。

3.11 关键记忆

- 图像具有局部关联性和一定不变性；CNN 的设计正是为了利用这些图像特征。p3-p6
- 图像不具有置换不变性；完全打乱像素顺序会破坏空间结构。p6
- 两个像素点完全独立时，相关函数或互信息量为 0。p3-p5
- 指数衰减下降较快，幂级数衰减下降较慢；幂级数衰减可以看作许多指数衰减成分的叠加。p3-p5
- 类视皮层设计启发 CNN：先提取局部简单特征，再逐步整合成复杂信息。p7-p10
- 卷积的核心是局部窗口加权求和；卷积核和 bias 都是可训练参数。p11-p13
- Padding 在边缘补值，用来控制输出空间大小；Same Padding 可保持空间尺寸不变。p14
- Stride 是卷积核滑动步长；stride 越大，输出空间尺寸通常越小。p15
- 多通道卷积中，一个完整卷积核覆盖所有输入通道；一个卷积核产生一个输出通道。p16
- 卷积输出空间尺寸看 H, W, K, P, S ；输出通道数看卷积核个数 C_{out} 。p14-p16
- 卷积参数量看 K, C_{in}, C_{out} ；有 bias 时再加 C_{out} ，不要乘 H_{out} 和 W_{out} 。p14-p16
- 池化汇总局部信息，降低空间尺寸，通常没有可学习参数，也通常不改变通道数；GAP 会把每个通道压成一个数。p18
- CNN 没有唯一固定格式；卷积、激活、池化、展平、全连接、softmax 等模块可以灵活组合，但相邻层的输入输出尺寸必须匹配。p20-p22

3.12 思考题

1. 判断：池化操作一定只能由专门的池化层实现，不能用卷积实现类似效果。
答案：错误。例如使用带 stride 的卷积可以在提取特征的同时降低空间尺寸，起到类似下采样的作用。
2. 判断：池化操作可以和卷积操作在同一层的设计中部分合并，例如用 stride 大于 1 的卷积完成下采样。
答案：正确。
3. 填空： 1×1 卷积核不改变单个位置的空间邻域大小，但可以改变 _____ 数。
答案：通道。
4. 计算：输入为 $32 \times 32 \times 3$ ，卷积核为 3×3 ，padding=1，stride=1，卷积核个数为 16，输出尺寸是 _____。
答案： $32 \times 32 \times 16$ 。
5. 计算：一个卷积层中 $K = 5$ ， $C_{in} = 3$ ， $C_{out} = 6$ ，且每个卷积核有 bias，参数量是 _____。
答案： $5 \times 5 \times 3 \times 6 + 6 = 456$ 。
6. 判断：全局平均池化 GAP 会把每个通道压成一个数，通常可以减少后续全连接层参数量。答案：正确。

4 循环神经网络 RNN 与 LSTM

这一章按 PDF 顺序复习：语言任务特点 -> 数据准备 -> 词元化 -> one-hot 与 embedding -> next token prediction -> RNN 设计 -> BPTT -> LSTM。重点是让学生理解：文本是可变长度序列，RNN 为什么适合处理序列，以及 LSTM 的门控机制解决了什么问题。

4.1 自然语言处理 NLP 的任务特点

页码：p3-p5

NLP 的目标是让计算机理解、生成和处理自然语言。常见任务包括机器翻译、文本分类、语音识别、信息检索、智能客服等。

自然语言有层次结构、语法结构、上下文相关性和长程依赖。更重要的是，文本天然是可变长度的离散符号序列。

自然语言处理 NLP：NLP 是让计算机理解、生成和处理人类自然语言的技术方向。

文本数据的可变长度：

文本和图像的一个关键区别是：文本序列长度通常不固定。不同句子的 token 数可能不同。

可变长度文本例子：

- 我喜欢 AI
- 我非常喜欢人工智能这门课
- 这部电影虽然节奏很慢，但是后半段非常精彩

这和图像任务很不一样。

图像与文本输入例子：图像通常可以统一 resize 成固定大小，例如 $32 \times 32 \times 3$ ；文本句子却天然长短不一，并且词语顺序会影响含义。

语言数据的几个特点：

特点	含义	例子
离散符号序列	文本由词、子词、字符等符号组成	“我 / 喜欢 / AI”
顺序重要	词语顺序会影响含义	“我打他”和“他打我”
上下文相关	当前词含义依赖上下文	“上海的交通大学”和“上海的交通”
长程依赖	较远位置的信息可能影响后面理解	前文主语影响后文指代

MLP、CNN 与 RNN 的区别：

模型	输入特点	是否天然适合可变长度序列
MLP	常需要固定长度向量	不适合
CNN	擅长局部窗口特征	不天然适合
RNN	逐 token 处理序列	适合

RNN 能处理可变长度文本的原因是：它不是一次性要求输入固定长度向量，而是按时间步逐个读入 token，并把已经读过的信息压缩到隐藏状态中。

RNN 处理可变长度序列的核心：RNN 按时间步逐个处理输入 token，并在不同时间步共享同一组参数；同一个 RNN 单元可以重复使用任意多次，因此可以处理不同长度的序列。

短句：x1 -> x2 -> x3

长句：x1 -> x2 -> x3 -> x4 -> x5 -> ...

同一个 RNN 单元可以在不同时间步重复使用。

4.2 数据准备

页码：p6-p7

数据准备是语言模型训练的基础。考试不考复杂数据工程流程，但要知道文本数据进入模型前通常需要清洗和处理。

步骤	作用
数据收集	获得足够数量的文本
质量过滤	去掉质量很差、无意义或噪声很大的文本
敏感内容过滤	减少有毒内容、隐私信息等风险
数据去重	减少重复样本，避免模型过度记忆重复文本

步骤	作用
词元化	把文本切成 token
向量化	把 token 变成神经网络能处理的数值向量

关键理解：数据不是直接进入 RNN。文本通常要先经过清洗、切分和向量化，才能变成神经网络输入。

一般来说，数据量增加有助于提升模型性能；数据质量高可以节约算力、增强稳定性、减少错误输出；大量重复数据可能让模型过度记忆重复模式，反而降低上下文处理能力。

4.3 分词、one-hot 与 embedding

页码：p8-p11

文本不能直接进入神经网络，通常要先切成 token，再变成向量。

文本 -> tokenization -> token 序列 -> one-hot 或 embedding 向量

Tokenization: Tokenization 是把原始文本切分成 token 序列的过程；token 可以是词、子词、字符或特殊符号。

常见词元化方法：

方法	记忆
基于规则	按空格、标点、词典等规则切分
BPE	从基本字符开始，逐步合并高频相邻片段
WordPiece	常见子词切分方法
Unigram	用统计模型选择合适的子词切分

例子 1: Tokenization.“我喜欢人工智能”可以切成词级 token：[我, 喜欢, 人工智能]；也可以切成子词级 token：[我, 喜欢, 人工, 智能]。

常见特殊符号：

特殊符号	常见含义
CLS	句首或分类标记
SEP	句尾或分隔标记
MASK	掩码标记
PAD	填充标记，用于补齐长度
UNK	未登录词或未知 token

Token 还需要变成数值向量，常见表示方式是 one-hot 和 embedding。

One-hot: One-hot 是用词表长度的稀疏向量表示一个 token，只有该 token 对应位置为 1，其余位置为 0。

Embedding: Embedding 是把 token 映射成低维稠密向量的表示方法，通常由可训练的 embedding 矩阵得到。

表示方式	特点	记忆
one-hot	高维、稀疏	不表达词义相似性
embedding	低维、稠密、可训练	可以学习语义关系

例子 2: One-hot 与 embedding。设词表为“我、喜欢、人工智能、这门课”，token “喜欢”的 one-hot 可以写成 $[0, 1, 0, 0]$ ，维度等于词表大小 4。如果 embedding 维度设为 3，它可能被映射为 $[0.12, -0.38, 0.51]$ 。

Embedding 通常由一个可训练矩阵得到，也可以理解为查表。设词表大小为 N ，embedding 维度为 d ：

$$\varepsilon_i \in R^N, \quad W_{\text{emb}} \in R^{d \times N}$$

$$x_i = W_{\text{emb}} \varepsilon_i, \quad x_i \in R^d$$

其中， ε_i 是第 i 个 token 的 one-hot 向量， W_{emb} 是可训练的 embedding 矩阵， x_i 是该 token 的 embedding 向量。

4.4 Next Token Prediction

页码：p13-p16

Next Token Prediction 是根据已有前文预测下一个 token。它是现代语言模型常用的训练框架。

Next Token Prediction：给定前文 token 序列，模型预测下一个 token；训练时每个位置的目标通常是输入序列中下一个位置的 token。

Next Token Prediction 是一种自监督训练方式：训练标签直接来自原始文本中的“下一个 token”，不需要额外人工标注。

Next Token Prediction 框架

语言模型的训练普遍采用 Next Token Prediction 框架，即输出序列的每一个 token 为输入序列的下一个 token



假设输入为： $X^{\text{in}} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$

模型的预测输出为： $\hat{X}^{\text{out}} = (\hat{\mathbf{x}}_2, \hat{\mathbf{x}}_3, \dots, \hat{\mathbf{x}}_{n+1})$

根据 Next Token Prediction，我们期望它的输出为： $X^{\text{out}} = (\mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_{n+1})$

于是可以对其计算交叉熵损失：

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n} \sum_{i=2}^{n+1} \sum_{j=1}^d x_{ij} \log \hat{x}_{ij}$$



大量文本都可以转化为“预测下一个 token”的任务：训练时，每个位置预测下一个 token；推理时，模型生成一个 token 后，把它接到上下文后继续生成。图中的 X^{in} 表示输入 token 序列，模型输出 $\hat{X}^{\text{out}}$ ，目标输出 X^{out} 是输入序列整体向后移动一位后的 token 序列。

推理时，模型每一步都会给出下一个 token 的概率分布。可以选择概率最大的 token（贪婪解码），也可以按概率分布采样得到不同输出；生成到终止符号（如 SEP）时可以停止。

训练时常用交叉熵损失，真实 token 的概率越高，loss 越小。图中给出的交叉熵损失可以写成：

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n} \sum_{i=2}^{n+1} \sum_{j=1}^d x_{ij} \log \hat{x}_{ij}$$

其中, x_{ij} 表示真实下一个 token 的 one-hot 标签, $\hat{x}_{ij}$ 表示模型预测的概率。

4.5 RNN 的设计

页码: p17-p18

RNN 是循环神经网络, 适合处理序列数据。它的核心设计是: 当前隐藏状态不仅看当前输入, 也看上一时刻隐藏状态。

循环神经网络 (Recurrent Neural Networks)

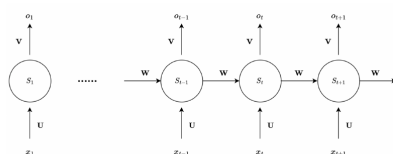


图 3.19: 循环神经网络的基本结构

如图3.19所示, 这是一个基本的循环神经网络单元, 下面我们详细介绍一下这个单元是如何工作的。我们的输入为时序序列 $x_1, x_2, \dots, x_{t-1}, x_t, x_{t+1}, \dots, x_n$, 并通过公式(3.15), (3.16)和(3.17)的方式计算得到对应的输出序列 $o_1, o_2, \dots, o_{t-1}, o_t, o_{t+1}, \dots, o_n$ 。

$$S_0 = 0, \quad (3.15)$$

$$S_t = f(\mathbf{U} \cdot x_t + \mathbf{W} \cdot S_{t-1} + b), \quad t = 1, 2, 3, \dots, n, \quad (3.16)$$

$$o_t = g(\mathbf{V} \cdot S_t) \quad (3.17)$$

其中 f 是激活函数, 通常使用 Tanh 函数, $g = \text{Softmax}(x)$ 是 softmax 函数。 S_t 是状态记忆, $\mathbf{U}$, $\mathbf{W}$, $\mathbf{V}$ 和 b 分别是输入权重矩阵、循环权重矩阵、输出权重矩阵和偏置项。



隐藏状态: RNN 的隐藏状态 S_t 是模型在第 t 步保存的历史信息摘要, 它由当前输入 x_t 和上一时刻隐藏状态 S_{t-1} 共同决定。

RNN 的隐藏状态可以理解为“目前为止读过内容的压缩记忆”。序列长度不同也没关系, 因为 RNN 可以重复使用同一个计算单元, 按时间步逐个处理输入。

考试要会识别 RNN 的基本公式:

$$S_0 = 0$$

$$S_t = f(Ux_t + WS_{t-1} + b), \quad t = 1, 2, \dots, n$$

$$o_t = g(VS_t + b_y)$$

其中:

设输入维度为 d_x , 隐藏状态维度为 d_h , 输出维度为 d_y :

符号	含义	形状
x_t	第 t 个输入向量	R^{d_x}
S_t	第 t 步隐藏状态	R^{d_h}
S_{t-1}	上一时刻隐藏状态	R^{d_h}
o_t	第 t 步输出	R^{d_y}
U	输入到隐藏状态的权重矩阵	$R^{d_h \times d_x}$
W	隐藏状态到隐藏状态的权重矩阵	$R^{d_h \times d_h}$

符号	含义	形状
V	隐藏状态到输出的权重矩阵	$R^{d_y \times d_h}$
b	隐藏状态偏置项	R^{d_h}
b_y	输出偏置项	R^{d_y}

f 常用 $\tanh$, g 可以是 softmax 。

RNN 参数量计算:

$$\#params = d_h d_x + d_h d_h + d_y d_h + d_h + d_y$$

其中, $d_h d_x$ 来自 U , $d_h d_h$ 来自 W , $d_y d_h$ 来自 V , d_h 来自隐藏状态偏置 b , d_y 来自输出偏置 b_y 。

RNN 参数量例子: 若输入维度 $d_x = 4$, 隐藏维度 $d_h = 3$, 输出维度 $d_y = 2$, 则

$$\#params = 3 \times 4 + 3 \times 3 + 2 \times 3 + 3 + 2 = 32$$

RNN 公式需要记住的三件事：

1. S_t 同时依赖当前输入 x_t 和历史状态 S_{t-1} 。
2. 参数 U, W, V 在不同时间步共享。
3. 输入多少个 token，就重复多少次同样的 RNN 单元，所以可以处理可变长度序列。

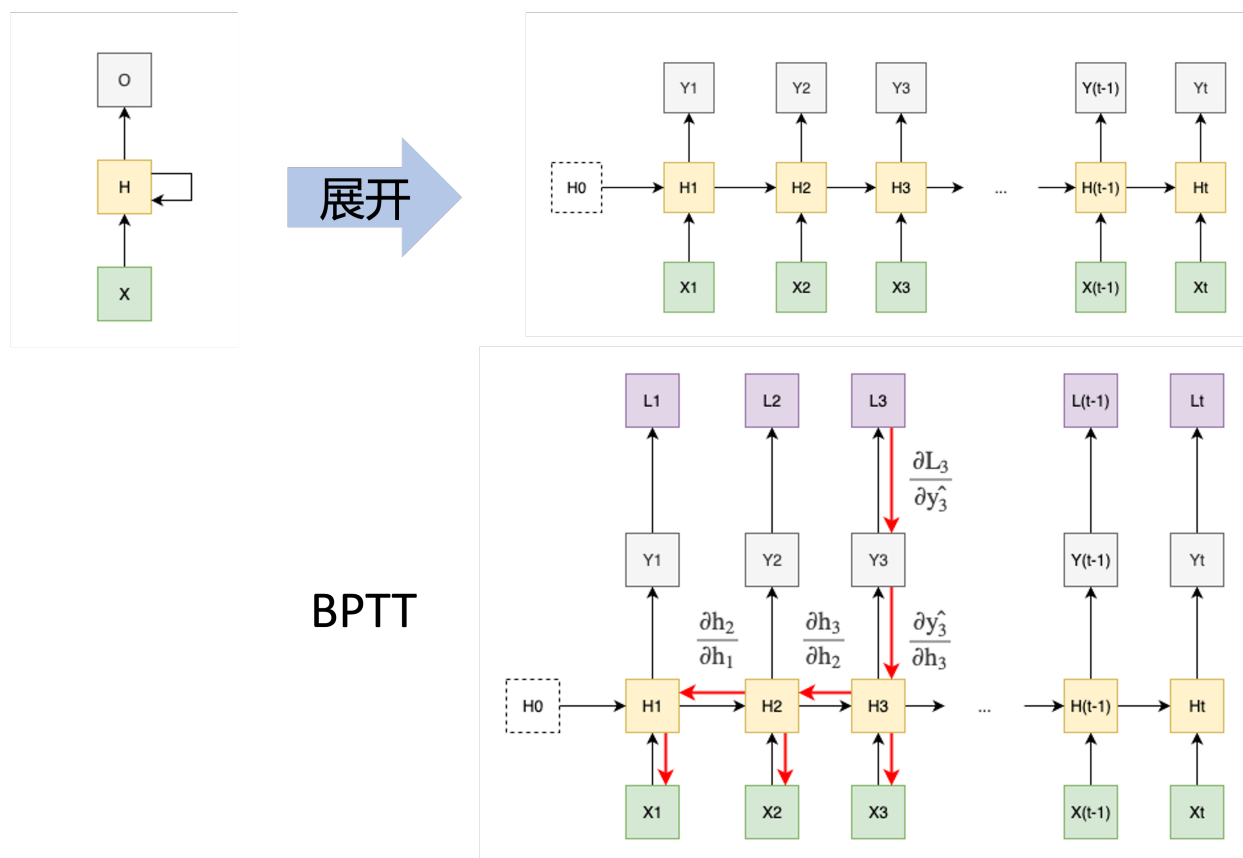
Encoder-Decoder RNN 可以处理输入输出长度不同的任务，例如机器翻译。

4.6 BPTT 与梯度问题

页码：p20

BPTT 是 BackPropagation Through Time，意思是随时间反向传播。训练 RNN 时，误差要沿多个时间步向前传播回去。

BPTT： BPTT 是随时间反向传播，把 RNN 按时间步展开后，沿时间方向反向计算梯度。



长序列中，梯度需要经过很多次链式相乘，容易出现两个问题：

问题	含义	影响
梯度消失	梯度越来越小	较早时间步的信息难以学习
梯度爆炸	梯度越来越大	训练不稳定

因此，普通 RNN 对长程依赖的建模能力有限。

如果序列非常长，理论上完整 BPTT 要展开并回传很多时间步，计算和显存开销都很大。实际训练中常用截断 BPTT，只向前回看有限步数，而不是对无限长历史完整反传。

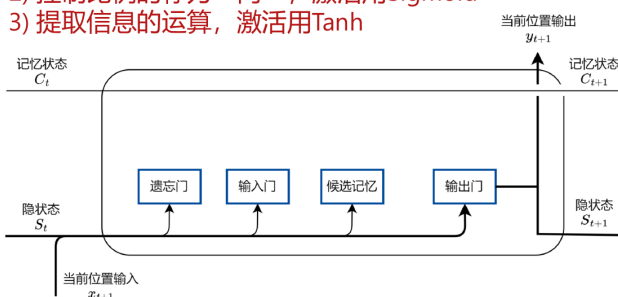
4.7 LSTM

页码: p21-p25

LSTM 是 Long Short-Term Memory, 长短期记忆网络。它在 RNN 基础上引入记忆状态, 并通过门控机制控制信息流动。

长短期记忆网络 (Long short-term memory, LSTM)

- 基本原则: 1) 任何信息使用前先做线性变换, 权重可学
2) 控制比例的称为“门”, 激活用 Sigmoid
3) 提取信息的运算, 激活用 Tanh



为什么能称为“遗忘门”、“输入门”、“输出门”、“知识”？
这是一种形象类比，没有严格证明。

门控神经元-遗忘门

保留的比例 $f_t = \sigma(W_{Sf}S_t + W_{xf}x_{t+1} + b_f)$

门控神经元-输入门

写入的比例 $i_t = \sigma(W_{Si}S_t + W_{xi}x_{t+1} + b_i)$

新的知识 $\tilde{C}_{t+1} = \tanh(W_{SC}S_t + W_{xC}x_{t+1} + b_C)$

新的记忆状态 $C_{t+1} = f_t * C_t + i_t * \tilde{C}_{t+1}$

门控神经元-输出门

输出的比例 $o_t = \sigma(W_{So}S_t + W_{xo}x_{t+1} + b_o)$

输出 $y_{t+1} = S_{t+1} = o_t * \tanh(C_{t+1})$

图 3.21: 单个门控神经元主要部件和并行隐藏状态示意图



LSTM: LSTM 是一种改进的 RNN, 它通过记忆状态和门控机制控制信息的保留、写入和输出, 用来缓解普通 RNN 的长程依赖问题。

结构	作用
记忆状态	保存较长期的信息
遗忘门	控制保留多少旧记忆
输入门	控制写入多少新信息
输出门	控制输出多少记忆信息

LSTM 三条设计原则: 信息使用前先做可学习的线性变换; 控制比例的门用 sigmoid; 提取候选信息常用 tanh。

门控机制: 门控机制用 0 到 1 之间的比例控制信息流动: 遗忘门控制旧记忆保留多少, 输入门控制新信息写入多少, 输出门控制当前输出多少记忆信息。

LSTM 和 Transformer 的一个共性是: 使用信息前通常都会先做可学习的线性变换, 用权重矩阵从 token 表示中提取需要的信息。

普通 RNN 与 LSTM 的区别可以概括为：

对比	普通 RNN	LSTM
保存历史信息	主要依靠隐藏状态 S_t	额外引入记忆状态 C_t
信息控制	缺少显式门控	用遗忘门、输入门、输出门控制信息比例
长程依赖	长序列中更容易梯度消失或梯度爆炸	通过记忆状态和门控机制缓解长程依赖
很长序列训练	常配合截断 BPTT	仍可配合截断 BPTT，但更容易保留长期信息

截断 BPTT 解决的是“序列太长，不能无限展开反传”的训练开销问题；LSTM 解决的是“普通 RNN 难以保留长期信息”的建模问题。LSTM 不能保证完全消除所有梯度问题，但它可以缓解普通 RNN 的长程依赖和梯度消失问题。

4.8 关键记忆

- NLP 处理的是离散符号序列，词语顺序和上下文都很重要。
- 文本数据通常是可变长度的；MLP/CNN 不天然适合记忆历史信息，RNN 通过隐藏状态逐步处理序列。
- 文本进入神经网络前通常要经过数据清洗、tokenization 和向量化。
- Tokenization 是把文本切成 token；token 可以是词、子词、字符或特殊符号。
- 常见词元化方法包括基于规则、BPE、WordPiece、Unigram；常见特殊符号包括 CLS、SEP、MASK、PAD、UNK。
- One-hot 高维稀疏，本身不表达词义相似性；embedding 低维稠密，可以通过训练学习语义关系。
- Next Token Prediction 是根据前文预测下一个 token，是自监督训练框架，标签来自文本中的下一个 token。
- RNN 的当前隐藏状态依赖当前输入和上一时刻隐藏状态，公式为 $S_t = f(Ux_t + WS_{t-1} + b)$ ；参数量计算要包含 U, W, V, b, b_y 。
- BPTT 是随时间反向传播；长序列中容易出现梯度消失或梯度爆炸。
- LSTM 通过记忆状态和门控机制控制信息保留、写入和输出，可以缓解长程依赖问题。

4.9 思考题

1. 判断：文本数据通常是可变长度序列，因此普通 MLP 不天然适合直接处理任意长度文本。
答案：正确。
2. 填空：把文本切成 token 的过程叫做 _____。
答案：tokenization 或词元化。
3. 填空：RNN 的基本公式中，当前隐藏状态可以写成 $S_t = f(Ux_t + W ______ + b)$ 。
答案： S_{t-1} 。
4. 判断：RNN 可以处理可变长度序列，一个重要原因是同一个 RNN 单元可以在不同时间步重复使用。
答案：正确。
5. 选择：LSTM 中控制保留多少旧记忆的是：A. 输入门 B. 遗忘门 C. 输出门 D. Softmax。
答案：B。
6. 计算：若 RNN 的输入维度 $d_x = 4$ ，隐藏维度 $d_h = 3$ ，输出维度 $d_y = 2$ ，参数量是多少？答案： $3 \times 4 + 3 \times 3 + 2 \times 3 + 3 + 2 = 32$ 。

5 Transformer

这一章按 PDF 顺序复习：为什么需要 Transformer -> next token prediction 回顾 -> 位置编码 -> 注意力机制与 Q/K/V -> causal mask -> Transformer block -> 输出层。重点是理解 Transformer 如何用注意力机制建模 token 之间的关系，以及为什么它比 RNN 更适合并行计算。

5.1 为什么需要 Transformer

页码：p3-p9

Transformer 出现前，序列任务常用 RNN/LSTM。RNN/LSTM 能处理序列，但有两个明显限制：长程依赖不容易保留，并且计算通常要按时间步顺序进行，不方便并行。

Attention is all you need!



Why Transformer?

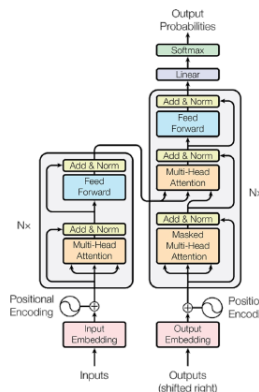
Transformer^[1] 诞生之前，在自然语言处理中大都采用基于RNN的编码器-解码器（Encoder-Decoder）结构来完成序列翻译。

RNN (LSTM) 的不足：

- 难以捕获长期依赖关系（梯度消失和梯度爆炸仍存在）

Transformer的优势：

- 更好地捕捉全局语义关系（多头注意力机制）
- 更好地理解上下文（位置编码）
- 更快的计算效率！（结构使得高效的并行计算成为可能）



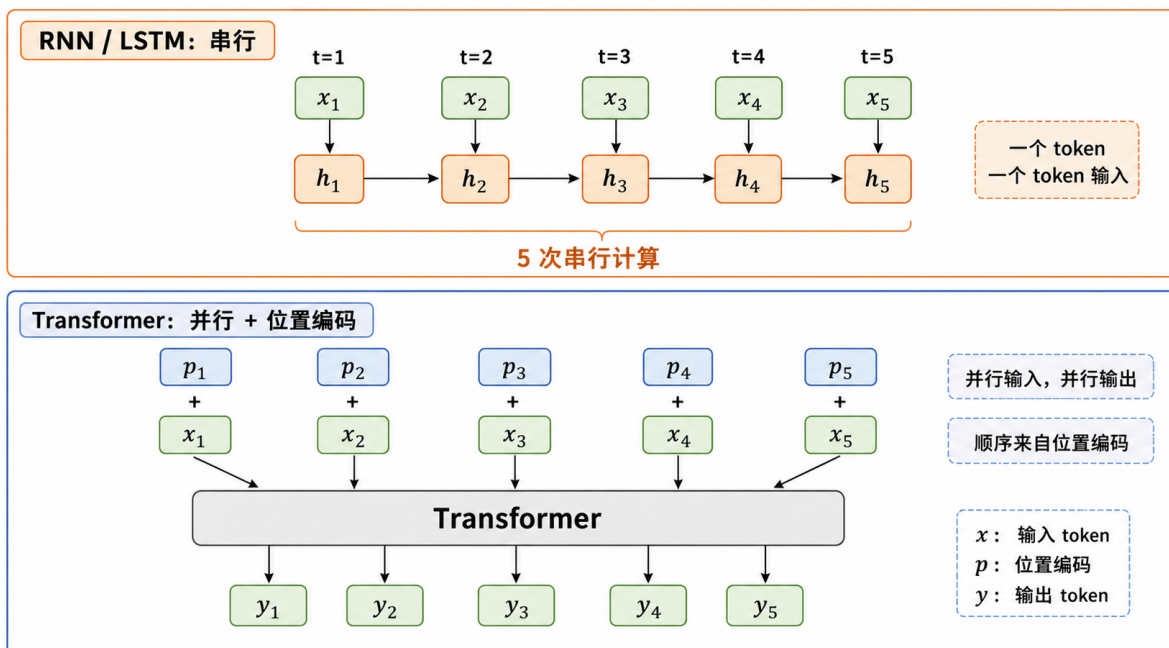
[1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.



Transformer： Transformer 是以注意力机制为核心的序列模型，能够直接建模 token 之间的关系，更容易捕捉全局上下文，也更适合并行计算。

Transformer 的直观思想可以理解为：当前 token 更新自己时，不是只看前一个隐藏状态，而是主动计算“我应该关注哪些 token”，再按关注程度汇总信息。

原始 Transformer 提出于 2017 年的论文《Attention is all you need》，最初用于机器翻译，采用 encoder-decoder 结构。现在常见的大语言模型多使用 decoder-only 结构，更适合 next token prediction 生成任务。



这张图要表达两件事：第一，RNN/LSTM 通过隐藏状态按时间步串行传递信息，5 个 token 通常要按时间顺序做 5 次串行计算；第二，Transformer 可以把一整段 token 同时送入模型，并行得到各位置输出，但 token 向量本身没有“第几个”的概念，因此必须额外加入位置编码。图中， x_i 表示第 i 个输入 token， p_i 表示第 i 个位置编码， y_i 表示第 i 个输出 token。

模型	处理序列的方式	主要特点
RNN/LSTM	按时间步顺序读入 token	有隐藏状态，但并行不方便，长程信息传递更困难
Transformer	直接计算 token 之间的关系	注意力机制更适合全局关系和并行计算

例子 1：注意力的直观类比。如果要估计某位同学缺失的化学成绩，可以先找“和他各科表现相似”的同学，再参考这些同学的化学成绩。注意力机制做的事情也类似：先计算相关性，再按相关性汇总信息。

5.2 Next Token Prediction 回顾

页码：p10-p13

Next token prediction 已经在“循环神经网络 RNN 与 LSTM”章的 4.4 节讲过：给定前文 token，模型预测下一个 token，并常用交叉熵损失。Transformer 这一章不重复展开，后面只需要记住：decoder 做生成任务时必须保证当前位置不能看到未来 token，因此需要 causal mask。

例子 2：回看 RNN 章内容。输入为“我爱小”，目标是预测下一个 token“猫”。Transformer 和 RNN 都可以放在 next token prediction 框架下训练，但 Transformer 用注意力机制处理上下文。

5.3 Embedding 与位置编码

页码：p15-p18

Embedding 已经在“循环神经网络 RNN 与 LSTM”章的 4.3 节讲过：token 先变成 one-hot，再通过可训练 embedding 矩阵映射成低维稠密向量。Embedding 矩阵通常随机初始化，并在训练过程中持续更新。本节重点是位置编码。

前面已经看到，Transformer 会把一组 token 同时送入模型。只有 embedding 还不够，因为 token embedding 只表示“这个 token 是什么”，不表示“这个 token 在第几个位置”。也就是说，如果只看一组 token 向量，模型并不知道谁在第 1 个位置、谁在第 2 个位置。位置编码用于告诉模型每个 token 在序列中的位置。

嵌入 Embedding + 位置编码



$$\begin{aligned}
 X^{\text{in}} &\in \mathbb{R}^{n \times d} \\
 X^{\text{em}} &= X^{\text{in}} W^{\text{em}} \in \mathbb{R}^{n \times d_m} \\
 W^{\text{em}} &\in \mathbb{R}^{d \times d_m} \quad \text{随机初始化，可训练} \\
 X^{\text{pos}} &\in \mathbb{R}^{n \times d_m} \\
 X^{(1)} &= X^{\text{em}} + X^{\text{pos}}
 \end{aligned}$$

X^{pos} : 可以是可学的参数，也可以是针对绝对位置或者相对位置的不可学习量。



位置编码：位置编码为模型提供 token 的位置信息；输入表示通常由 token embedding 和位置编码相加得到。位置编码不改变词表大小。

输入表示：

$$X^{(1)} = X^{emb} + X^{pos}$$

其中, $X^{emb} \in R^{n \times d_m}$ 是 token embedding, $X^{pos} \in R^{n \times d_m}$ 是位置编码, 因此 $X^{(1)} \in R^{n \times d_m}$ 。 n 是序列长度, d_m 是每个 token 的向量维度。

例子 3: 为什么需要位置编码。“我 爱 小 猫”和“猫 爱 小 我”包含相同 token, 但顺序不同, 含义也不同。位置编码就是告诉 Transformer: 这些 token 分别出现在第几个位置。

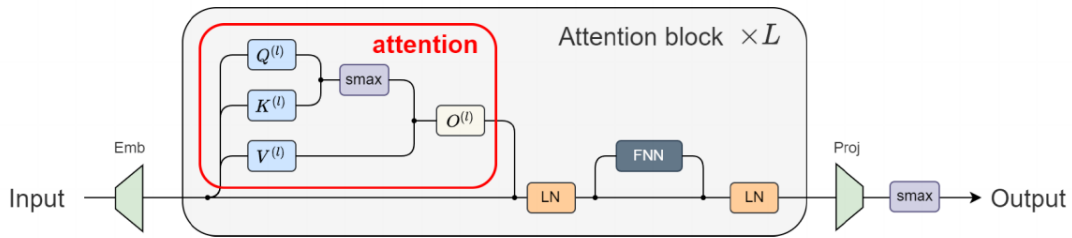
课件后面的复杂位置编码属于阅读材料, 复习时不展开推导。考试重点是知道: Transformer 需要位置信息, 输入表示通常是 embedding 加位置编码。

5.4 注意力机制与 Q、K、V

页码: p21-p31

注意力主要计算 token 之间的相关性, 本身不负责记录“谁在第几个位置”, 所以输入 Transformer 前要把位置编码加进去。

注意力机制



$$\begin{aligned} Q^{(l)} &= X^{(l)} W^{Q(l)}, W^{Q(l)} \in R^{d_m \times d_k} \\ K^{(l)} &= X^{(l)} W^{K(l)}, W^{K(l)} \in R^{d_m \times d_k} \\ V^{(l)} &= X^{(l)} W^{V(l)}, W^{V(l)} \in R^{d_m \times d_v} \end{aligned}$$

l 表示层的指标



Q、K、V: Q 是 Query, 表示当前 token 发出的查询; K 是 Key, 表示每个 token 可被匹配的键; V 是 Value, 表示真正被加权汇总的信息内容。

总公式: 缩放点积注意力

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

这条公式分四步理解:

1. 线性映射得到 Q, K, V 。输入是 $X^{(l)} \in R^{n \times d_m}$, 其中 n 是 token 数量, d_m 是每个 token 的隐藏向量维度。 Q, K, V 不是新的输入数据, 而是由同一个 $X^{(l)}$ 通过三个可训练线性映射得到:

$$\begin{aligned} Q^{(l)} &= X^{(l)} W^{Q(l)}, & W^{Q(l)} &\in R^{d_m \times d_k} \\ K^{(l)} &= X^{(l)} W^{K(l)}, & W^{K(l)} &\in R^{d_m \times d_k} \\ V^{(l)} &= X^{(l)} W^{V(l)}, & W^{V(l)} &\in R^{d_m \times d_v} \end{aligned}$$

输出为 $Q^{(l)}, K^{(l)} \in R^{n \times d_k}$, $V^{(l)} \in R^{n \times d_v}$ 。Q 和 K 的最后一维必须相同，V 的维度可以不同。一般注意力中，Query 的个数可以和 Key/Value 的个数不同；self-attention 中三者通常来自同一段序列，所以 token 数相同。

2. 用 Q 和 K 计算关联强度。输入是 $Q^{(l)}, K^{(l)} \in R^{n \times d_k}$ ：

$$A^{(l)} = \frac{Q^{(l)}(K^{(l)})^T}{\sqrt{d_k}}, \quad A^{(l)} \in R^{n \times n}$$

A_{ij} 表示第 i 个 token 对第 j 个 token 的关注分数；除以 $\sqrt{d_k}$ 是为了让训练更稳定。

3. 对关联强度做 softmax 得到注意力权重。输入是 $A^{(l)} \in R^{n \times n}$ ：

$$\text{Attn}^{(l)} = \text{softmax}(\text{mask}(A^{(l)})), \quad \text{Attn}^{(l)} \in R^{n \times n}$$

mask 是可选遮挡步骤，生成任务里的 causal mask 见 5.5 节；如果不需要遮挡，可以理解为直接对 $A^{(l)}$ 做 softmax。

4. 用注意力权重对 V 加权求和，再用 W^O 调整维度。输入是 $\text{Attn}^{(l)} \in R^{n \times n}$, $V^{(l)} \in R^{n \times d_v}$ ：

$$X^{qkv(l)} = \text{Attn}^{(l)} V^{(l)}, \quad X^{qkv(l)} \in R^{n \times d_v}$$

再通过输出矩阵 $W^{O(l)}$ 映射回模型维度：

$$X^{pr(l)} = X^{qkv(l)} W^{O(l)}, \quad W^{O(l)} \in R^{d_v \times d_m}$$

最终输出 $X^{pr(l)} \in R^{n \times d_m}$ 。 W^O 的作用是把 $n \times d_v$ 变回 $n \times d_m$ ，这样才能和原输入 $X^{(l)}$ 做残差相加。

例子 4: 注意力加权求和。如果某个 token 对三个位置的注意力权重是 $[0.7, 0.2, 0.1]$ ，三个 value 分别是 $[1, 3, 5]$ ，则汇总结果为

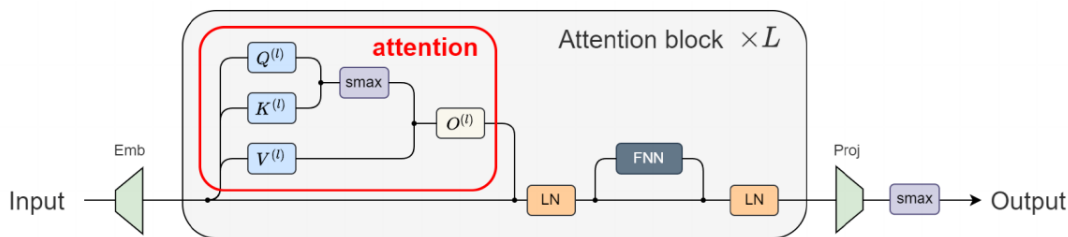
$$0.7 \times 1 + 0.2 \times 3 + 0.1 \times 5 = 1.8$$

5.5 Causal Mask

页码：p27-p29

在生成任务中，预测当前位置时不能看到未来 token，否则模型就相当于提前看到了答案。Causal mask 用来遮住未来位置。

注意力机制



算关联+Mask+归一化

除 $\sqrt{d_k}$ 是为训练稳定

$$\text{Attn}^{(l)} = \text{softmax} \left(\frac{\text{mask}(Q^{(l)}(K^{(l)})^T)}{\sqrt{d_k}} \right) \in \mathbb{R}^{n \times n}$$

$$\text{softmax}(A)_{ij} = \frac{e^{a_{ij}}}{\sum_{j=1}^m e^{a_{ij}}}$$

$$\begin{bmatrix} 0 & 0 & \cdots & \cdots & 0 \\ & 0 & \cdots & 0 & 0 \\ & & 0 & \cdots & \\ & & & 0 & 0 \\ & & & & 0 \end{bmatrix}$$

元素 ij : 第 i 个词对第 j 个词的注意力值
只能对自己前面的词有关注!



Causal Mask: Causal mask 会把未来位置的注意力分数设为很小的值，使 softmax 后这些位置的权重接近 0，从而保证预测时不能看未来 token。

对第 i 个 token 来说，causal mask 只允许它关注第 1 到第 i 个 token。这样就保证 next token prediction 的因果性。

例子 5: Causal mask 允许看哪些位置。对序列“我 爱 小 猫”，每个位置能关注的位置如下：

当前位置	允许关注	不能关注
我	我	爱、小、猫
爱	我、爱	小、猫
小	我、爱、小	猫
猫	我、爱、小、猫	无

对应的 causal mask 可以写成一个矩阵。行表示当前位置，列表示被关注的位置；0 表示允许看， $-\infty$ 表示遮住：

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

计算时可以理解为先把 mask 加到注意力分数上：

$$\text{Attn} = \text{softmax}(A + M)$$

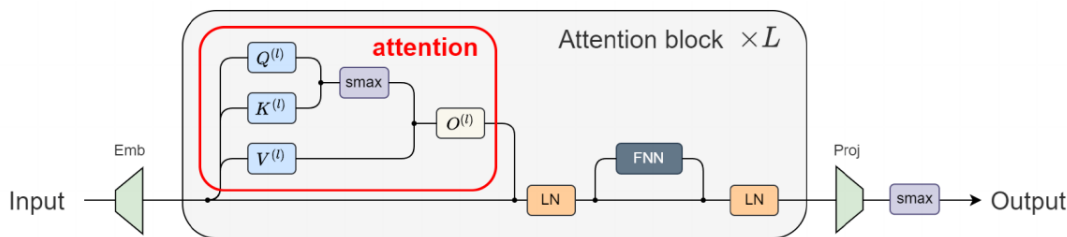
因此，当模型在“我 爱”之后预测下一个 token 时，可以利用“我”和“爱”，但不能提前看到“小”或“猫”。

5.6 Transformer Block、LayerNorm 与 FNN

页码: p31-p37

Transformer block 可以理解为“先让 token 之间交流，再分别处理每个 token”。多个 block 可以重复堆叠。

注意力机制



算关联+Mask+归一化+加权求和+线性变换

$$X^{\text{pr}(l)} = X^{\text{qkv}(l)} W^{O(l)} \in \mathbb{R}^{n \times d_m}, \quad W^{O(l)} \in \mathbb{R}^{d_v \times d_m}$$

$$X^{\text{ao}(l)} = \text{LayerNorm}(X^{(l)} + X^{\text{pr}(l)}) \in \mathbb{R}^{n \times d_m}$$

$$[\text{LayerNorm}(X)]_{i,j} = \gamma * \frac{X_{i,j} - \sum_l X_{i,l}/d_m}{\text{std}(X_{i,1}, \dots, X_{i,d_m})} + \beta. \quad \gamma \text{ 和 } \beta \text{ 可训练}$$



Transformer Block: 一个 Transformer block 通常包含注意力模块、残差连接、LayerNorm 和前馈全连接网络 FNN。

Block 中四个核心部件:

1. 注意力模块: 负责 token 之间的信息交互。注意力输出经 W^O 后变回 $n \times d_m$, 才能和输入做残差相加。
2. 残差连接: 把子层输入直接加到子层输出上, 基本形式是

$$y = x + F(x)$$

3. LayerNorm: 逐个 token 做归一化。对 $n \times d_m$ 的矩阵, 它对每一行 token 向量内部的 d_m 个数计算均值和方差, 不在不同 token 之间混合。归一化后还有可训练仿射参数 γ, β 。
4. FNN: 逐 token 作用的小 MLP; 不同位置共享同一套 FNN 参数。FNN 不负责 token 之间的信息交互, token 之间的 interaction 主要发生在 attention 里。

LayerNorm 公式示意:

$$X^{\text{ao}(l)} = \text{LayerNorm}(X^{(l)} + X^{\text{pr}(l)})$$

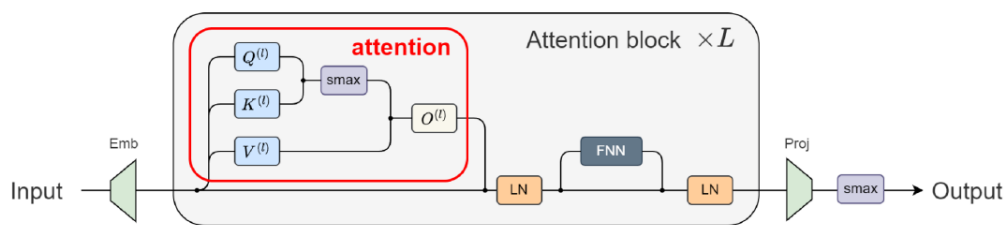
例子 6: LayerNorm 和 FNN 都是逐 token。如果输入有 4 个 token, 每个 token 是 128 维向量, LayerNorm 会分别对 4 个 token 各自的 128 个数做归一化; FNN 也分别作用在 4 个 token 上。不同 token 之间的信息交换主要已经在注意力模块中完成。

5.7 输出投影与解码

页码: p38-p39

最后, 输出投影层把每个 token 的隐藏向量映射回词表大小, 再经过 softmax 得到下一个 token 的概率分布。

输出 (投影) 层



$$X^{\text{out}} = X^{\text{do}(L)} W^{\text{proj}} + b^{\text{proj}} \in \mathbb{R}^{n \times d}, \quad W^{\text{proj}} \in \mathbb{R}^{d_m \times d}, \quad b^{\text{proj}} \in \mathbb{R}^d$$

贪婪解码 (greedy decoding) $\text{Output} = \text{argmax}(\text{softmax}(X^{\text{out}}))$

采样解码 (sampling decoding) $\text{Output} \sim \text{softmax}(X^{\text{out}})$



输出投影层：输出投影层把隐藏向量映射到词表维度，softmax 后得到每个候选 token 的概率。

输出公式：

$$X^{\text{out}} = X^{\text{do}(L)} W^{\text{proj}} + b^{\text{proj}}, \quad W^{\text{proj}} \in \mathbb{R}^{d_m \times d}$$

其中， d 是词表大小，所以输出的最后一维对应词表中每个 token 的分数。

语言模型训练通常用交叉熵损失衡量预测分布和真实下一个 token 之间的差距；真实 token 的预测概率越高，loss 越小。

解码方式	含义	结果特点
贪婪解码	每一步取概率最大的 token	确定性强；同一输入通常得到同一输出
采样解码	按概率分布随机抽样	有随机性；同一输入可能得到不同输出

例子 7：贪婪解码和采样解码的区别。假设下一个 token 的概率为：猫 0.70、狗 0.20、鼠 0.10。贪婪解码一定选择概率最大的“猫”；采样解码大多数时候会抽到“猫”，但也有可能抽到“狗”或“鼠”。所以采样解码更有随机性。

5.8 关键记忆

- Transformer 用注意力机制直接建模 token 之间的关系，比 RNN 更适合并行计算。p3-p9
- 原始 Transformer 是 encoder-decoder 结构；现在常见生成式大语言模型多用 decoder-only。p3-p9
- RNN/LSTM 按时间步串行处理 token；Transformer 可以并行处理 token，但需要位置编码提供顺序信息。p3-p18
- Embedding 矩阵随机初始化并可训练；位置编码提供 token 位置信息，不改变词表大小。p15-p18
- 注意力公式是 $\text{softmax}(QK^T/\sqrt{d_k})V$ ；Q 是查询，K 是匹配用的键，V 是被汇总的信息。p21-p31
- Q 和 K 的最后一维必须相同，V 的维度可以不同；注意力权重形状通常是 $n \times n$ 。p21-p31
- Causal mask 保证预测当前位置时不能看到未来 token。p27-p29
- Block 中 attention 负责 token 交互；FNN 逐 token 作用；LayerNorm 逐 token 计算，并有可训练参数 γ, β 。p31-p37
- 残差连接公式是 $y = x + F(x)$ ，用于保留原信息并缓解深层训练困难。p31-p37
- 输出投影层把隐藏向量映射到词表维度；贪婪解码取最大概率 token，采样解码具有随机性。p38-p39
- Transformer 整体流程：输入 token -> Embedding + 位置编码 -> 多层 block -> 输出投影 -> Softmax。p15-p39
- 多头注意力是了解项：多组注意力并行工作，不同头可以关注不同类型的信息。p21-p31

5.9 思考题

1. 判断：Transformer 使用注意力机制来建模 token 之间的关系。
答案：正确。
2. 填空：Transformer 需要加入 _____，用来表示 token 在序列中的位置。
答案：位置编码。
3. 选择：Q、K、V 中，用来真正汇总信息内容的是：A. Q；B. K；C. V；D. mask。
答案：C。
4. 判断：Q 和 K 的最后一维必须相同，因为要计算 QK^T 。
答案：正确。
5. 判断：LayerNorm 是把不同 token 混在一起计算均值和方差。
答案：错误。LayerNorm 是逐 token 在向量内部计算。
6. 填空：缩放点积注意力的核心公式可以写成 $\text{softmax}(QK^T/\sqrt{d_k})$ _____。答案：V。

6 神经网络训练现象：频率原则

这一章按 PDF 顺序复习：训练过程现象 -> 泛化谜团 -> 隐式偏好 -> 频率与傅里叶级数 -> 频率原则 -> 泛化与早停。重点是理解“神经网络通常先学低频、后学高频”这个现象，以及它为什么和泛化有关。后续多尺度网络、Fourier feature、NeRF、理论证明和激活函数频谱属于扩展/阅读材料，不作为复习重点。

6.1 从训练现象出发

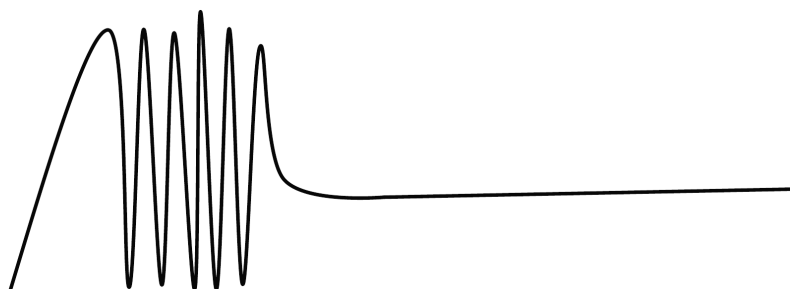
页码：p5-p8

神经网络训练过程并不是“所有部分同时学好”。对一个既有平坦区域、又有振荡区域的函数，训练时通常更容易先学好平坦、缓慢变化的部分，振荡和细节往往更晚才学好。

哪一部分先学习好



左边是振荡，右边是平坦



低频部分：变化慢、比较平滑，通常对应整体轮廓或主要趋势。

高频部分：变化快、振荡明显，通常对应细节、边缘、纹理或噪声。

例子 1：先学轮廓，再学细节。画一把梳子时，通常会先画外轮廓，再画密集的梳齿；神经网络拟合函数时也常有类似现象：平滑轮廓更容易先被学到，密集振荡更难、更晚被学到。

6.2 泛化谜团与隐式偏好

页码：p9-p11

过参数化神经网络参数很多，理论上可以表达非常复杂的函数，甚至可以把训练样本“记住”。但实际训练中，神经网络常常并不会明显过拟合，而是得到泛化较好的解。这就是课件中提出的泛化谜团。

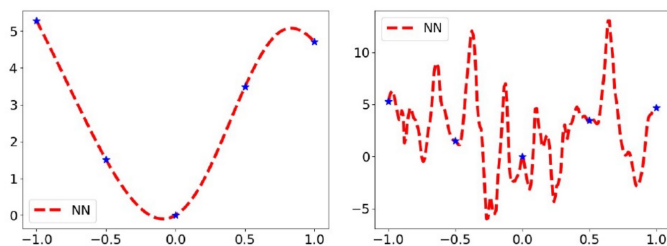
过参数化：模型参数量相对样本数很多，模型有能力表达大量不同的函数。

隐式偏好：当许多解都能让训练误差很低甚至为 0 时，训练过程本身会倾向于得到某些类型的解；这种偏向没有显式写在损失函数里。

选择哪种解？



蓝色训练点是完全一致的。



常用设定学到的解，
满足频率原则

参数初始值很大时学到的解，
实验观察其不满足频率原则



这张图要表达的是：蓝色训练点完全一致时，可能存在不止一种拟合曲线。有的解更平滑，有的解振荡更强。频率原则可以帮助解释为什么普通训练常更倾向平滑、低频的解。

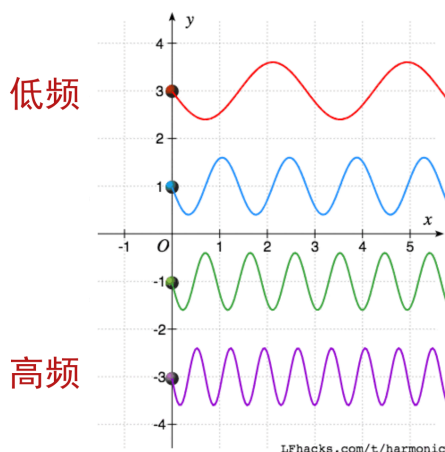
例子 2：同样训练点，多种解。如果训练点很少，一条平滑曲线和一条剧烈振荡的曲线都可能穿过所有训练点。训练误差看起来都很好，但平滑曲线通常更符合“整体规律”，更可能在未见过的位置上预测得合理。

6.3 频率、低频与高频

页码：p12-p16

频率用来描述函数变化的快慢。低频表示变化慢、整体趋势明显；高频表示变化快、细节多。复习时不需要推导傅里叶级数，只需要知道：复杂函数可以从“不同频率成分叠加”的角度来理解。

傅里叶级数



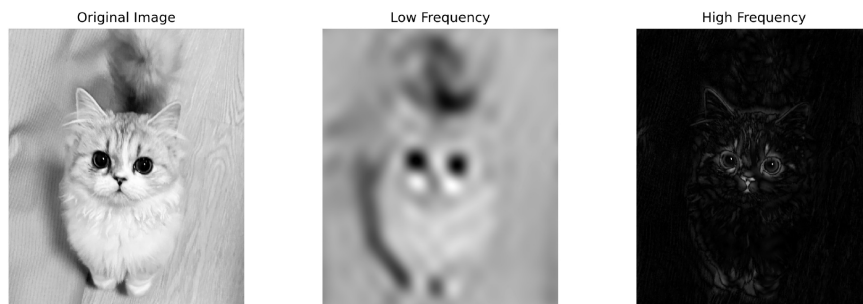
频率：频率描述函数或信号变化的快慢；变化越慢，频率越低；变化越快，频率越高。

傅里叶级数：

$$f(x) \approx a_0 + a_1 \sin x + b_1 \cos x + a_2 \sin 2x + b_2 \cos 2x + \dots$$

可以把这个式子理解为：一个复杂函数可以近似看成许多不同频率的正弦、余弦函数叠加。 a_0 表示整体平均水平，后面的项表示不同频率成分及其强度。

保留低频或高频的图像变化



在图像中，低频通常保留整体形状和大轮廓，高频通常保留边缘、纹理和细碎变化。高频并不是一定没用，但噪声也常表现为高频。

例子 3：函数中的低频和高频。 $f_1(x) = \sin x$ 变化较慢，可以看成低频； $f_2(x) = \sin 8x$ 在同样区间内振荡更多，可以看成更高频。

例子 4：图像中的低频和高频。一张人脸图像的模糊轮廓更偏低频；头发纹理、边缘和噪声点更偏高频。

6.4 频率原则

页码：p17-p22

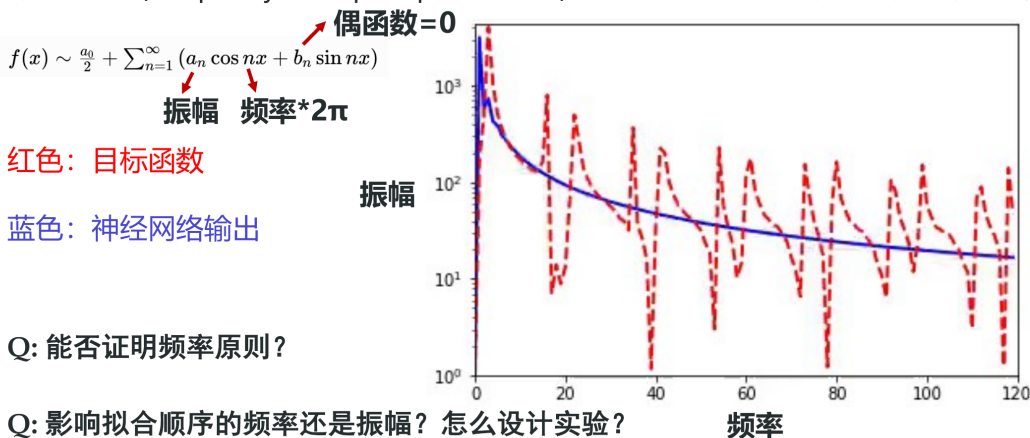
课件的核心结论是：神经网络训练过程中，通常先拟合低频成分，再逐渐拟合高频成分。这个现象叫频率原则，也叫 spectral bias。

频率原则：神经网络在训练过程中通常按照从低频到高频的顺序拟合目标函数。

频域空间一维函数训练过程



频率原则 (Frequency Principle/Spectral bias) : 神经网络按照从低频至高频的顺序拟合



从时域看: 训练早期先出现整体轮廓, 训练后期才逐渐补上细节。

从频域看: 低频成分的误差通常先下降, 高频成分的误差通常后下降。

控制变量实验: 如果把不同频率的振幅控制得一样, 仍然观察到低频先被拟合, 就说明拟合顺序不只是由“振幅大小”决定, 频率本身也很重要。

例子 5: 一维函数训练。对目标函数

$$f(x) = \sin x + \sin 5x$$

$\sin x$ 是低频成分, $\sin 5x$ 是更高频成分。训练时通常会先学到 $\sin x$ 对应的平滑轮廓, 再逐渐学到 $\sin 5x$ 对应的振荡细节。

6.5 频率原则与泛化

页码: p23-p26

频率原则可以帮助理解泛化: 很多函数都能穿过训练点, 但普通训练更倾向于先得到低频、平滑的解。这样的解通常不容易把训练样本中的偶然噪声全部记住, 因此更可能在测试数据上表现合理。

泛化: 模型不仅在训练样本上表现好, 也能在没有见过的新样本上表现好。

需要注意: 频率原则描述的是常见训练倾向, 不是说高频永远没有用, 也不是说神经网络永远不会学到高频。训练足够久、模型足够强时, 高频细节甚至高频噪声也可能被学到。

6.6 Early Stopping 早停

页码: p27-p28

数据中可能含有噪声, 而噪声常常偏高频。由于神经网络通常先学低频信号、后学高频细节, 训练太久可能开始拟合高频噪声, 导致测试误差上升。早停就是在模型进一步拟合噪声前停止训练。

Early Stopping: 早停是在验证误差或测试误差开始变差前后停止训练, 用来减少过拟合、提升泛化能力。

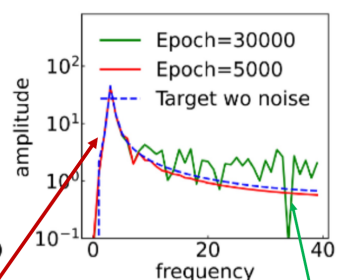
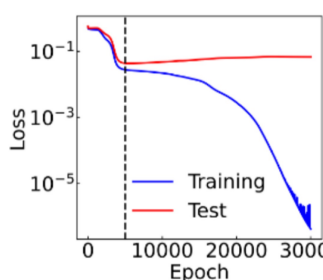
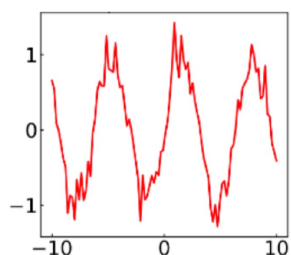
Early stopping (早停) 的效用



数据从 $\sin x$ 中采样，但有噪音！

讨论：为什么测试误差经过一段时间后开始上升？

蓝色是没有噪音的目标函数



主要低频几乎不受影响

噪音影响高频

早停可以有效防止模型拟合高频噪声，提高模型泛化能力



例子 6：带噪声的正弦函数。目标整体趋势接近 $\sin x$ ，但样本中混入了抖动噪声。训练早期模型先学到 $\sin x$ 的整体趋势；如果继续训练太久，模型可能开始拟合噪声。早停可以让模型保留主要趋势，同时少学一些高频噪声。

6.7 图像分类中的两种频率

页码：p37-p40

图像分类里说“高频”时容易混淆两件事：一种是图像本身的像素频率，另一种是分类函数的响应频率。考试更容易考这个区别。

图像频率：图像频率描述相邻像素强度变化的快慢；边缘、纹理和细节通常偏高频，平缓背景和整体轮廓通常偏低频。

响应频率：响应频率描述输入发生微小变化时，模型输出变化的敏感程度；响应频率高表示很小的输入扰动也可能引起输出明显变化。

图像分类问题中，频率原则讨论的重点通常是分类函数的响应频率，而不是图像像素内部的频率。对抗噪声就是一个典型例子：图片只加入肉眼几乎看不见的小扰动，模型预测类别却发生改变，说明扰动方向上分类函数响应很敏感，具有明显高频成分。

例子 7：不要混淆两种频率。一张猫图像的毛发纹理属于图像频率里的高频；如果给图片加很小的扰动，模型输出从“猫”变成“狗”，这反映的是分类函数对该扰动方向的响应频率高。

6.8 关键记忆

- 低频表示变化慢、平滑、整体轮廓；高频表示变化快、振荡、细节或噪声。p5-p16
- 过参数化模型有能力表达很多复杂函数，但神经网络训练常能得到泛化较好的解。p9-p11
- 隐式偏好指训练过程会偏向某些解，这种偏向没有显式写在损失函数里。p10-p11
- 频率原则也叫 spectral bias：神经网络通常先拟合低频，再拟合高频。p17-p22
- 频率原则可以解释为什么神经网络常先学整体规律，而不是一开始就记住高频噪声。p23-p28
- Early stopping 可以在模型进一步拟合高频噪声前停止训练，从而改善泛化。p27-p28
- 高频不是一定没用；边缘、纹理、细节可能是高频，噪声也可能是高频。p12-p16
- 图像分类中的频率更常指分类函数的响应频率，不是图像像素内部的频率。p37-p40

6.9 思考题

1. 判断：频率原则指神经网络训练时通常先拟合低频成分，再拟合高频成分。
答案：正确。
2. 填空：频率原则也叫 spectral _____。
答案：bias。
3. 选择：图像中的边缘、纹理和噪声通常更接近：A. 低频；B. 高频；C. 标签；D. bias。
答案：B。
4. 判断：过参数化神经网络参数很多，所以一定会严重过拟合，不可能泛化好。
答案：错误。
5. 判断：早停可以理解为在模型进一步拟合高频噪声前停止训练。
答案：正确。
6. 判断：图像分类中讨论频率时，一定指图像像素内部的频率，与模型输出无关。答案：错误。图像分类中更常指分类函数的响应频率。

7 参数凝聚与解的平坦性

这一章按 PDF 顺序复习：初始化大小 -> ReLU 转折点 -> 参数凝聚 -> 为什么不直接训练小网络 -> 平坦解与训练技巧。重点是理解“小初始化下网络可能出现凝聚”，以及“平坦解为什么通常更稳健”。

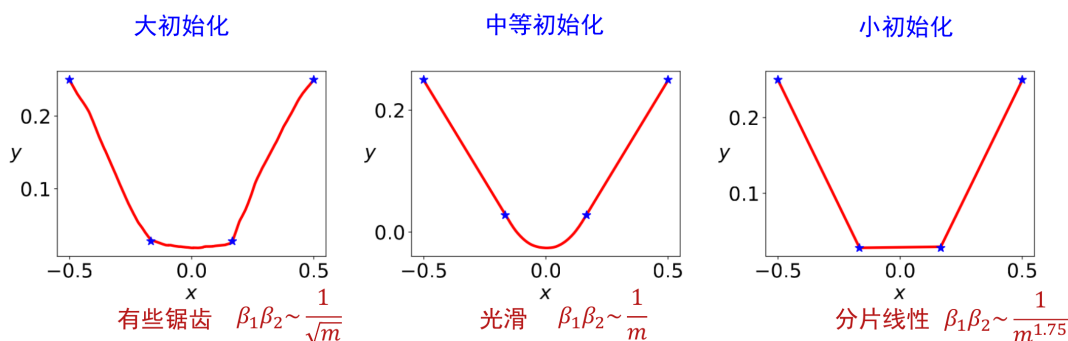
7.1 初始化大小如何影响训练

页码：p4-p8

初始化不是一个纯技术细节，它会影响网络一开始处在参数空间的什么位置，也会影响训练早期的输出形态和后续优化路径。

不同初始化大小对网络输出的影响

神经网络模型： $f(x) = \sum_{k=1}^{1000} a_k \sigma(w_k x + b_k)$ ，激活函数： $\sigma(x) = \text{ReLU}(x) = \max(0, x)$



通过分析一维函数拟合的特点，来推断一般任务是如何受初始化影响



初始化：初始化是在训练开始前给网络参数赋初值；初始化大小会影响训练速度、训练路径和最终解的形态。

需要掌握的结论：

- 大初始化：初始输出更随机、更复杂，容易保留较多随机振荡。
- 中等初始化：通常是训练稳定性和表达能力之间的折中。
- 小初始化：初始输出更平滑，神经元更容易出现趋同或凝聚。
- 常见初始化方法包括 LeCun、Xavier、Kaiming，本质上都是在控制不同网络结构下的初始化尺度。
- 所有参数不能都初始化为 0，否则同层神经元完全相同，梯度更新也相同，无法有效打破对称性。

例子 1：为什么不能全 0 初始化。如果同一层两个神经元的权重和 bias 都初始化成完全相同的值，它们接收相同输入、产生相同输出、得到相同梯度，训练后仍然很像，相当于没有真正训练出两个不同特征。

7.2 ReLU 神经元与转折点

页码：p9-p12

ReLU 神经元的非线性主要体现在“从关闭到打开”的转折位置。理解转折点，有助于理解后面的神经元凝聚。

参数不同会导致ReLU的转折点不同



$$\sigma(x) = \text{ReLU}(x) = \max(0, x)$$



- ReLU函数的转折点：

$$wx + b = 0$$

$$x = -\frac{b}{w}$$

- 每个神经元的非线性主要在**转折点附近**

$$k > 0, \sigma(k(wx + b)) = k \max(wx + b)$$



ReLU 转折点：对一维输入， $\text{ReLU}(wx + b)$ 的转折点满足 $wx + b = 0$ ，即 $x = -\frac{b}{w}$ 。

公式需要会用：

$$z = wx + b, \quad \text{ReLU}(z) = \max(0, z)$$

$$wx + b = 0 \quad \Rightarrow \quad x = -\frac{b}{w}$$

两个 ReLU 神经元如果转折点相同、方向相同，就可能合并成一个等效神经元；如果转折点不同，一般不能简单合并。

例子 2：转折点计算。对 $\text{ReLU}(2x - 4)$ ，转折点满足 $2x - 4 = 0$ ，所以 $x = 2$ 。对 $\text{ReLU}(x - 2)$ ，转折点也是 $x = 2$ ，但输出斜率不同。

7.3 参数凝聚

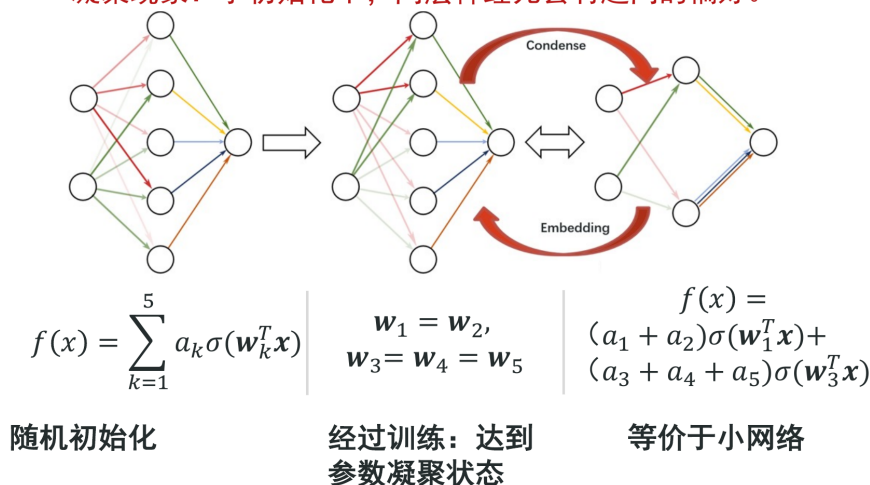
页码：p13-p23

小初始化下，同一层的多个神经元可能逐渐变得相似，最后表现得像少数几个“有效神经元”。这就是参数凝聚。

小初始化的凝聚现象示意图



凝聚现象：小初始化下，同层神经元会有趋同的偏好。



参数凝聚：小初始化下，同层神经元会出现趋同偏好，多个神经元可能等效为较少的有效神经元。

需要掌握的结论：

- 凝聚不是“网络坏了”，而是许多神经元在参数空间中靠近，表现出相似作用。
- 凝聚后，大网络在函数表达上可能等效于一个更小的网络。
- 有效参数量可以理解为等效小网络真正使用到的参数量。
- 训练过程中，模型可能先用较少有效神经元拟合主要结构，再逐渐增加有效神经元学习更复杂的部分。
- 这与频率原则可以联系起来：有效神经元变多后，网络更有能力学习更细、更高频的变化。

例子 3：相似神经元可以合并。如果三个神经元方向和转折点都相同：

$$a_1 \sigma(wx + b) + a_2 \sigma(wx + b) + a_3 \sigma(wx + b) = (a_1 + a_2 + a_3) \sigma(wx + b)$$

它们可以看成是一个更大的等效神经元。

7.4 为什么不直接训练小网络

页码：p24

如果凝聚后的大网络可以等效为小网络，直觉上会问：为什么不一开始就训练小网络？

需要掌握的回答：

- 从表达结果看：凝聚后的大网络和等效小网络可能表达同一个函数。
- 从训练过程看：大网络参数更多，下降方向更多，通常更容易优化。
- 小网络虽然参数少，但可选择下降方向少，训练时可能更难找到合适解。
- 小初始化有助于出现凝聚，但初始化过小也可能让训练变慢；默认初始化通常是实用折中。

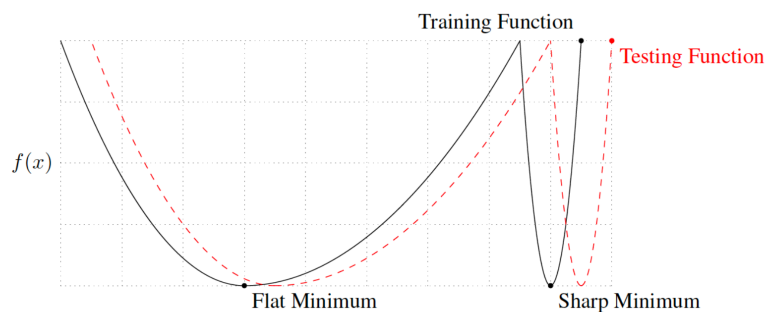
例子 4：表达能力相同，训练难度不同。两个相似神经元合并后可以表示同一个函数，但训练时保留两个神经元相当于多了可调节的方向，优化更灵活。

7.5 平坦解与尖锐解

页码: p25-p30

神经网络训练得到的不只是一个“训练误差低”的点，还要看这个点附近的损失变化。附近损失也低，说明解更稳定。

平坦解与尖锐解



讨论：选择哪个解比较好？为什么？

Y轴为损失函数

训练集和测试集通常都会有噪音，平坦的解更鲁棒，泛化好。

X轴为参数

Keskar, Nitish Shirish, et al. "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima." International Conference on Learning Representations, 2016.



平坦解：参数在一个小范围内扰动时，损失仍然保持较低的解。

尖锐解：参数稍微扰动，损失就明显上升的解。

需要掌握的结论：

- 训练集和测试集通常都有噪声，平坦解对扰动更鲁棒。
- 平坦解通常泛化更好，但不表示训练误差一定更低。
- 尖锐解对参数变化更敏感，测试表现可能更不稳定。
- 小批量训练带有随机性，通常更容易找到较平坦的解；大批量训练更可能停在较尖锐的区域。

7.6 提升平坦性的常见技巧

页码: p31-p33, p37

这一部分只需要掌握结论：训练设置会影响模型更容易落在平坦解还是尖锐解附近。

- 小批次找到的解更平坦
- 学习率越（合理的）大，找到的解越平坦
- Dropout算法找到解更平坦

常见有助于平坦性的做法：

- 使用小批量训练：梯度噪声更大，更可能逃离尖锐区域。
- 使用合理较大的学习率：有助于跳出过尖锐的局部区域，但过大会震荡甚至发散。
- 使用 Dropout：训练时随机屏蔽部分神经元，使模型不依赖单一路径，可能得到更鲁棒的解。

7.7 关键记忆

- 初始化大小会影响训练路径和最终解；参数不能全部初始化为 0。p4-p8
- 对一维输入， $\text{ReLU}(wx + b)$ 的转折点是 $x = -\frac{b}{w}$ 。p9-p12
- 参数凝聚指小初始化下，同层神经元出现趋同偏好，多个神经元可能等效为较少有效神经元。p13-p23
- 凝聚后的大网络可能像小网络，但大网络参数更多、下降方向更多，通常更容易训练。p24
- 平坦解指参数附近一片区域损失都较低；尖锐解指参数稍微改变损失就明显上升。p25-p30
- 平坦解通常对噪声和参数扰动更鲁棒，泛化往往更好。p25-p30
- 小批量训练、合理较大的学习率、Dropout 都可能帮助找到更平坦的解。p31-p37

7.8 思考题

1. 计算： $\text{ReLU}(2x - 4)$ 的转折点是 _____。
答案： $x = 2$ 。
2. 判断：所有参数初始化为 0 通常不是好做法，因为同层神经元会保持相同，难以打破对称性。
答案：正确。
3. 填空：小初始化下，同层神经元可能出现趋同偏好，这种现象叫 _____。
答案：参数凝聚。
4. 判断：凝聚后的大网络和等效小网络可能表达同一个函数，但大网络通常更容易训练。
答案：正确。
5. 选择：平坦解通常指：A. 参数附近一片区域损失都较低；B. 参数必须全为 0；C. 训练误差一定为 1；D. 没有梯度。
答案：A。
6. 判断：学习率越大越好，越大一定容易得到平坦解。
答案：错误。合理较大的学习率可能有帮助，但过大会导致震荡甚至发散。

8 大模型介绍

8.1 什么是大模型

复习重点：

1. 大模型通常也叫基础模型。
2. 大模型参数量很大。
3. 大模型经过大规模数据训练。
4. 大模型可以适应多种下游任务。
5. 大语言模型是最早展现出通用智能特征的大模型之一。
6. 许多模态的数据也可以类似语言一样进行 token 化。

易错点：

1. “大模型通常经过大规模数据训练。”正确。
2. “大模型只能做一个固定任务。”错误。
3. “基础模型可以适应多种下游任务。”正确。

8.2 Encoder、Decoder 与常见架构

复习重点：

1. Encoder 可以看到上下文所有 token。
2. Decoder 只能看到当前位置和前文 token。
3. Encoder-only 常用于理解任务。
4. Decoder-only 常用于生成任务。
5. Encoder-Decoder 常用于输入和输出都需要建模的任务，如翻译。
6. BERT 是 Encoder-only 的代表。
7. GPT、Llama、DeepSeek 等是 Decoder-only 的代表。
8. BART、T5 是 Encoder-Decoder 的代表。
9. 当前主流生成式大语言模型多采用 Decoder-only。

对照表：

架构	是否看未来	常见任务	代表
Encoder-only	可以看双向上下文	理解任务	BERT
Decoder-only	只能看当前和前文	生成任务	GPT/Llama/DeepSeek
Encoder-Decoder	编码输入，解码输出	翻译等	T5/BART

易错点：

1. “Decoder-only 在生成任务中很常见。”正确。
2. “Encoder-only 非常适合直接做 next token prediction。”通常不适合。
3. “BERT 是 Decoder-only。”错误。

8.3 预训练、微调、提示词、蒸馏

复习重点：

1. 预训练是在大规模数据上训练基础能力。
2. 语言模型预训练常用 next token prediction。
3. 微调是在预训练模型基础上，用特定任务数据继续训练。
4. 强化学习后训练可增强推理能力，也可用于对齐人类偏好。
5. 提示词工程通过设计输入让模型给出更好的回答。
6. 知识蒸馏把大模型能力迁移到小模型中。
7. Teacher model 是提供知识的大模型或强模型。
8. Student model 是接受蒸馏的小模型或目标模型。

易错点：

1. “预训练通常需要大量数据。”正确。
2. “提示词工程一定会更新模型参数。”错误。
3. “知识蒸馏可以让小模型学习大模型输出中的知识。”正确。

8.4 大模型中的现象

复习重点：

1. Scaling law 描述模型规模、数据量、计算量和测试损失之间的规律关系。
2. 一般来说，在一定范围内，规模、数据、算力增加会带来性能提升。
3. 上下文学习指模型不更新参数，仅通过任务描述和示例适应任务。
4. 思维链是让模型生成中间推理步骤。
5. 思维链可提升复杂推理任务表现。
6. 幻觉是模型生成看似真实但实际错误或误导的信息。
7. 大模型可能包含偏见。

易错点：

1. “上下文学习不需要更新模型权重。”正确。
2. “思维链就是引导模型写出中间推理步骤。”正确。
3. “大模型永远不会产生幻觉。”错误。
4. “大模型可能带有偏见。”正确。

8.5 思考题

1. 判断：大模型通常经过大规模数据训练，可以适应多种下游任务。
答案：正确。
2. 选择：当前主流生成式大语言模型多采用：A. Encoder-only B. Decoder-only C. 纯 CNN D. 纯 RNN。
答案：B。
3. 填空：预训练是在大规模数据上训练模型的 _____ 能力。
答案：基础。
4. 判断：上下文学习通常不更新模型参数，而是通过任务描述和示例让模型适应任务。
答案：正确。
5. 判断：幻觉指模型生成看似真实但实际错误或误导的信息。
答案：正确。

9 强化学习

这一章是考试重点，但不建议考复杂推导。重点是概念、关系和最简单公式。

9.1 强化学习是什么

复习重点：

1. 强化学习是决策型任务。
2. 智能体在环境中采取动作。
3. 环境返回奖励和新的状态。
4. 智能体通过反复试错学习。
5. 目标是最大化期望累计奖励。
6. 奖励可以是正的，也可以是负的。
7. 强化学习不同于监督学习：通常没有每一步的正确标签，而是通过奖励反馈学习。

基本循环：

状态 s_t $\rightarrow$ 智能体选择动作 a_t $\rightarrow$ 环境给出奖励 r_{t+1} 和下一状态 s_{t+1}

易错点：

1. “强化学习只做静态分类任务。”错误。
2. “强化学习强调智能体和环境交互。”正确。
3. “奖励只能是正数。”错误。

9.2 状态、观察、动作空间、奖励

复习重点：

1. 状态 State 是环境完整信息。
2. 观察 Observation 是智能体实际看到的信息。
3. 完全可观测环境中，观察和状态一致。
4. 部分可观测环境中，观察只是状态的一部分。
5. 动作空间是所有可选动作的集合。
6. 动作空间可以是离散的，也可以是连续的。
7. 棋类、超级马里奥常可看作离散动作空间。
8. 自动驾驶中的方向盘角度、油门等常可看作连续动作空间。
9. 奖励是评价动作好坏的反馈信号。

易错点：

1. “观察一定等于完整状态。”错误。
2. “动作空间可以是离散的，也可以是连续的。”正确。
3. “奖励用于告诉智能体行为好坏。”正确。

9.3 回报与折扣因子

复习重点：

1. 强化学习不只关心当前奖励，还关心未来奖励。
2. 回报是未来奖励的累积。
3. 折扣因子 γ 用于降低远期奖励权重。
4. γ 的取值范围是 $0 \leq \gamma \leq 1$ 。
5. γ 越接近 0，越重视眼前奖励。
6. γ 越接近 1，越重视长期奖励。

无折扣回报：

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

例题：未来三个奖励分别为 1, 0, 2, $\gamma = 0.5$ 。

$$G_t = 1 + 0.5 \times 0 + 0.5^2 \times 2 = 1.5$$

易错点：

1. “强化学习完全不关心未来奖励。”错误。
2. “折扣因子通常在 0 到 1 之间。”正确。
3. “ γ 越接近 1，越重视长期奖励。”正确。

9.4 情节型任务与连续型任务

复习重点：

1. 情节型任务有起点和终点。
2. 一局游戏、一次棋局、一个关卡通常是情节型任务。
3. 连续型任务没有固定终止状态。

4. 股票交易、交通控制、自动驾驶可看作连续型任务。

易错点：

1. “所有强化学习任务都有明确终点。”错误。
2. “棋类游戏通常可以看作情节型任务。”正确。
3. “连续型任务需要持续决策。”正确。

9.5 探索与利用

复习重点：

1. 探索 Exploration 是尝试未知或不确定动作以获取信息。
2. 利用 Exploitation 是根据已有知识选择当前看起来最好的动作。
3. 只利用可能错过更好的策略。
4. 只探索可能无法获得足够高奖励。
5. 强化学习需要平衡探索和利用。

生活例子：每天去熟悉但还不错的餐厅是利用；尝试一家没去过的新餐厅是探索。

易错点：

1. “探索是尝试未知动作。”正确。
2. “利用是根据已有知识做当前较优选择。”正确。
3. “强化学习中完全不需要探索。”错误。

9.6 策略、价值函数与最优策略

复习重点：

1. 策略 policy 是智能体选择动作的规则。
2. 策略可以是确定性的，也可以是概率性的。
3. 价值函数 $V(s)$ 衡量从状态 s 出发能获得的期望回报。
4. 状态价值高，说明从这个状态出发更有希望获得高累计奖励。
5. 最优策略是在长期意义下使期望累计奖励最大的策略。
6. 最优价值函数和最优策略相互关联。

易错点：

1. “策略就是状态到动作选择的规则。”正确。
2. “价值函数只看当前一步奖励，完全不看未来。”错误。
3. “最优策略追求最大化期望累计奖励。”正确。

9.7 MDP 与马尔可夫性

复习重点：

1. MDP 是 Markov Decision Process，马尔可夫决策过程。
2. MDP 是强化学习常用数学模型。
3. 马尔可夫性指未来只依赖当前状态和动作，而不依赖完整历史。
4. 如果当前状态已经包含足够信息，就不需要再记住所有过去信息。
5. MDP 常包含状态、动作、转移概率、奖励、折扣因子等元素。

必背句：

马尔可夫性：给定当前状态和动作后，未来状态的概率分布与更早历史无关。

易错点：

1. “MDP 是马尔可夫决策过程。”正确。
2. “马尔可夫性要求未来依赖全部历史。”错误。

3. “当前状态如果足够完整，就可以用于预测下一状态分布。”正确。

9.8 Bellman 方程的直观含义

复习重点：

1. Bellman 方程刻画当前价值和未来价值的递推关系。
2. 当前状态的价值等于当前奖励加上折扣后的下一状态价值的期望。
3. Bellman 方程是许多强化学习算法的基础。
4. 不要求推导复杂公式，只要理解“当前价值 = 当前收益 + 未来价值”。

直观形式：

当前价值 = 当前奖励 + γ × 未来价值

不考虑动作时，向量形式：

$$V = R + \gamma P V$$

易错点：

1. “Bellman 方程把当前价值和未来价值联系起来。”正确。
2. “Bellman 方程完全不考虑未来。”错误。
3. “Bellman 方程是强化学习中的重要基础。”正确。

9.9 DP、MC 与深度强化学习

复习重点：

1. 动态规划 DP 可以利用 Bellman 方程迭代求价值。
2. DP 的优点是可以较精确计算价值。
3. DP 的缺点是通常需要知道环境转移概率 P 。
4. 蒙特卡洛 MC 通过多次实验采样取平均。
5. MC 的优点是简单直接，不一定需要明确知道转移概率。
6. MC 的缺点是随机性大，方差可能大。
7. 深度强化学习把深度神经网络引入强化学习。
8. 深度神经网络可以近似价值函数或策略。
9. DQN 用神经网络替代表格来拟合 Q 值。
10. 深度强化学习能处理高维状态，如图像输入。
11. 深度强化学习也带来训练不稳定、数据需求大、算力需求高等问题。

对照表：

方法	核心思想	优点	缺点
DP	用 Bellman 方程迭代	可较精确计算	需要环境模型/转移概率
MC	多次采样取平均	简单直接	随机性大，方差可能大
DRL	用深度网络近似价值或策略	处理高维复杂状态	训练难、数据和算力需求大

易错点：

1. “DP 通常需要知道转移概率。”正确。
2. “MC 通过采样估计价值。”正确。
3. “DQN 用神经网络近似 Q 值。”正确。
4. “深度强化学习完全没有训练困难。”错误。

9.10 强化学习应用

复习重点：

1. 强化学习可用于游戏 AI。
2. 强化学习可用于机器人控制。
3. 强化学习可用于自动驾驶、交通灯调度、推荐系统、数据中心节能等。
4. 大模型后训练中也可使用强化学习。
5. 在 ChatGPT 一类系统中，可以用人类偏好构造奖励信号。
6. 在推理模型中，强化学习可用于增强推理能力。

易错点：

1. “强化学习只能玩游戏。”错误。
2. “强化学习可以用于机器人控制。”正确。
3. “大模型后训练中可能使用强化学习。”正确。

9.11 思考题

1. 判断：强化学习中，智能体通过与环境交互，并根据奖励反馈学习。
答案：正确。
2. 填空：强化学习的目标通常是最大化期望 _____ 奖励。
答案：累计。
3. 计算：未来三个奖励为 1, 0, 2, $\gamma = 0.5$, 折扣回报为 _____。
答案： $1 + 0.5 \times 0 + 0.5^2 \times 2 = 1.5$ 。
4. 选择：尝试一个不确定但可能更好的动作，属于：A. 探索 B. 利用 C. 监督学习 D. 归一化。
答案：A。
5. 判断：马尔可夫性指给定当前状态和动作后，未来状态分布不再依赖更早历史。
答案：正确。

10 最后一遍串讲清单

10.1 残差网络必会

1. 深层普通网络可能出现退化问题。
2. 退化问题指网络加深后训练误差或测试误差反而变差。
3. 深层网络还可能遇到梯度消失和梯度爆炸。
4. 残差连接的基本形式是 $F(x) + x$ 。
5. 残差连接帮助信息和梯度在深层网络中传播。
6. ResNet 主要用于缓解深层网络训练困难和退化问题。

10.2 CNN 必会

1. 图像具有局部关联性和一定不变性。
2. 关联性会随距离衰减，课件中有指数衰减和幂级数衰减两类模型。
3. 幂级数衰减可以理解为许多指数衰减成分的叠加，长程关联更不容易完全消失。
4. CNN 与 MLP 的核心区别是局部连接和参数共享。
5. 卷积核在图像上滑动，用局部窗口提取边缘、方向、纹理等局部特征。
6. 卷积核参数通常随机初始化，然后训练得到。
7. RGB 输入有 3 个通道；输出通道数由卷积核个数决定。
8. 会根据输入大小、卷积核大小、padding、stride 计算输出空间尺寸；输出通道数等于卷积核个数。
9. 会根据卷积核大小、输入通道数、卷积核个数计算卷积层参数量。
10. 池化可以降低空间尺寸，逐渐处理更长范围的关联，通常不改变通道数。
11. CNN 可由卷积、激活、池化、展平、全连接、输出层等模块按任务需要组合。

10.3 RNN/LSTM 必会

1. NLP 是处理自然语言的任务。

2. 语言有顺序、上下文、长程依赖。
3. Tokenization 把文本切分成 token。
4. One-hot 稀疏且不表达语义相似。
5. Embedding 可训练，能学习词向量。
6. Next Token Prediction 是预测下一个 token。
7. RNN 能处理序列，因为有历史状态。
8. BPTT 训练 RNN，长序列可能梯度消失/爆炸。
9. LSTM 通过记忆状态和门控机制缓解长程依赖问题。

10.4 Transformer 必会

1. Transformer 核心是注意力机制。
2. Embedding 把 token 变向量。
3. 位置编码提供顺序信息。
4. Q/K/V 分别是 Query/Key/Value。
5. Q 和 K 算相关性，V 被加权求和。
6. Softmax 把注意力分数变成权重。
7. Causal mask 防止 Decoder 看未来。
8. LayerNorm 逐 token 归一化。
9. FNN 逐 token 作用。
10. 输出层映射到词表大小。

10.5 训练现象必会

1. 频率原则：先低频后高频。
2. 低频是轮廓、平滑、整体趋势。
3. 高频是细节、振荡、快速变化，也可能包含噪声。
4. 早停可以减少拟合高频噪声。
5. 初始化影响训练。
6. 小初始化可能导致参数凝聚。
7. 参数凝聚使多个神经元趋向相似偏好。
8. 平坦解通常更鲁棒、泛化更好。

10.6 大模型必会

1. 大模型又叫基础模型。
2. 大模型通常参数量大、数据规模大、适应任务多。
3. Encoder-only 常用于理解任务。
4. Decoder-only 常用于生成任务。
5. 当前主流生成式大语言模型多是 Decoder-only。
6. 预训练学习基础能力。
7. 微调用特定任务数据继续训练。
8. 提示词工程改变输入，不一定改参数。
9. 知识蒸馏把大模型知识迁移到小模型。
10. 上下文学习不更新权重。
11. 思维链生成中间推理步骤。
12. 大模型可能产生幻觉和偏见。

10.7 强化学习必会

1. 强化学习是决策型任务。
2. 基本循环：状态、动作、奖励、下一状态。
3. 目标是最大化期望累计奖励。
4. 状态是完整环境信息，观察是智能体看到的信息。
5. 动作空间可以离散或连续。
6. 折扣因子 γ 在 0 到 1 之间。

7. γ 越大越重视长期奖励。
8. 情节型任务有终点，连续型任务没有固定终点。
9. 探索是尝试未知，利用是使用已有知识。
10. 策略是选择动作的规则。
11. 价值函数衡量状态的期望回报。
12. MDP 是马尔可夫决策过程。
13. 马尔可夫性：未来只依赖当前状态和动作。
14. Bellman 方程联系当前价值和未来价值。
15. DP 需要环境模型，MC 通过采样估计。
16. 深度强化学习用神经网络近似价值函数或策略。

11 不建议作为考试重点的内容

以下内容可以课堂提到，但不建议作为选择、判断、填空题重点。

1. 卷积平移等变性的严格证明。
2. Pooling 反向传播细节。
3. LSTM 门控公式完整推导。
4. 正余弦位置编码、RoPE、相对位置编码的公式推导。
5. 多头注意力复杂矩阵维度计算。
6. 多尺度神经网络、Fourier 特征、NeRF。
7. Hessian、稳定边缘的数学推导。
8. LoRA、MCP、Agent、RAG 的技术细节。
9. TD、Sarsa、Q-learning 的公式推导。
10. Kaggle 注册、实验提交、代码复现操作。

12 两节复习课建议讲法

12.1 第一节课：网络结构主线

建议顺序：

1. 5 分钟：说明考试范围和“不考阅读材料”。
2. 10 分钟：残差网络，重点讲退化问题、梯度传播困难和残差连接。
3. 30 分钟：CNN，按“图像数据特点 -> 为什么需要卷积 -> 卷积/padding/stride/池化计算 -> 经典结构”讲。
4. 20 分钟：RNN/LSTM，重点讲序列、BPTT、梯度问题、门控机制。
5. 25 分钟：Transformer，重点讲 embedding、位置编码、QKV、mask、输出层。

课堂建议：

1. CNN 部分一定要带学生完整算 2-3 个尺寸题。
2. Transformer 部分不要推矩阵公式，重点讲 Q/K/V 的角色。
3. RNN/LSTM 部分不要推门控公式，重点讲为什么需要记忆和门。

12.2 第二节课：训练现象、大模型、强化学习

建议顺序：

1. 20 分钟：频率原则、早停、初始化、凝聚、平坦解。
2. 20 分钟：大模型架构、预训练、微调、提示词、蒸馏、scaling law、上下文学习、思维链。
3. 35 分钟：强化学习基本概念、回报计算、探索利用、MDP、Bellman、DP/MC/DRL。
4. 15 分钟：串讲易错点。

课堂建议：

1. 训练现象部分用“先学轮廓再学细节”来讲频率原则。
2. 大模型部分重点做概念辨析，不要展开最新技术细节。
3. 强化学习部分要反复画“状态-动作-奖励-下一状态”的循环。

4. Bellman 方程只讲“当前价值 = 当前奖励 + 折扣未来价值”。